

ĐẠI HỌC QUỐC GIA, THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC ỨNG DỤNG



NHẬP MÔN KỸ THUẬT

BÁO CÁO BTL
PHÁT HIỆN VẾT NỨT TRÊN XI MĂNG
QUA HÌNH ẢNH SỬ DỤNG
MẠNG NƠ-RON TÍCH CHẬP (CNN)

Giáo viên hướng dẫn: Cô Phan Thị Khánh Vân
Thầy Lê Xuân Đại

Thành viên	MSSV
Nguyễn Bùi Gia Tiến	2413479
Lê Nguyễn Minh Tân	2413084
Đỗ Lê Chí Kiên	2411749
Nguyễn Minh Trí	2413637
Đỗ Phương Nghi	2412245



Nhận xét của GVHD:

Mục lục

1	LỜI CẢM ƠN	3
2	TỔNG QUAN ĐỀ TÀI	4
3	CƠ SỞ LÝ THUYẾT	5
3.1	Giới thiệu	5
3.2	Tổng quan về mạng nơ-ron tích chập (CNN)	5
3.2.1	CNN là gì?	5
3.2.2	Lý do CNN phù hợp với phân tích hình ảnh.	5
3.3	Kiến trúc cơ bản của CNN	6
3.3.1	Các thành phần chính	6
3.3.2	Luồng xử lý dữ liệu trong CNN.	11
3.4	Các ưu điểm và hạn chế của CNN	13
3.5	Nguyên lý hoạt động của CNN trong phân tích hình ảnh	13
3.5.1	Cách CNN xử lý hình ảnh	13
3.5.2	Ví dụ minh họa	14
3.5.3	Các ứng dụng thực tế của CNN	18
3.5.4	Thách thức và hướng phát triển	18
3.5.5	Kết luận	19
4	ĐOẠN CODE KAGGLE	20
4.1	Nhập các thư viện cần thiết	20
4.2	Thiết lập các thông số chung	20
4.3	Tải dữ liệu, tiền xử lý và trực quan hóa	20
4.4	Lập mô hình sử dụng CNN	26
4.4.1	Chi tiết các lớp trọng mô hình CNN	28
4.4.2	Tóm tắt hoạt động của mô hình	29
4.5	Đánh giá mô hình CNN	33
5	LỜI KẾT	44



1 LỜI CẢM ƠN

Báo cáo có thể được xem là một dự án khoa học nhỏ. Do vậy để hoàn thành một đề tài báo cáo là một việc không dễ dàng đối với sinh viên chúng em. Chúng em xin bày tỏ lòng biết ơn chân thành và sâu sắc đến Thầy Lê Xuân Đại, người đã dùng những tri thức và tâm huyết của mình để có thể truyền đạt cho chúng em vốn kiến thức quý báu. Bên cạnh đó, nhóm chúng em cũng xin cảm ơn Cô Phan Thị Khánh Vân đã giúp đỡ và hướng dẫn chúng em tận tình trong suốt thời gian viết bài báo cáo này, tạo cho chúng em những tiền đề, những kiến thức để tiếp cận, phân tích giải quyết vấn đề đang gặp phải.

Thành công luôn đi kèm với nỗ lực, trong vòng nhiều tuần, nghiên cứu đề tài “*Phát hiện vết nứt trên xi măng qua hình ảnh sử dụng mạng nơ-ron tích chập (CNN)*” chúng em cũng đã gặp không ít khó khăn, thử thách nhưng nhờ có sự giúp đỡ của cô chúng em đã vượt qua. Chúng em đã cố gắng vận dụng những kiến thức đã học được trong học kỳ qua để hoàn thành bài báo cáo này nhưng do chưa có nhiều kinh nghiệm làm đề tài cũng như những hạn chế về kiến thức nên chắc chắn sẽ không tránh khỏi những thiếu sót. Chúng em rất mong nhận được sự nhận xét, ý kiến đóng góp, phê bình từ phía cả hai thầy cô để bài báo cáo được hoàn thiện hơn.

Chúng em cũng xin cảm ơn bạn bè, anh chị đã tận tình chỉ bảo chúng em trong quá trình hoàn thành bài báo cáo, tạo điều kiện cho chúng em hiểu thêm về những kiến thức thực tế.

Một lần nữa, nhóm chúng em xin cảm ơn Thầy Cô vì đã giảng dạy và trang bị kiến thức cần thiết để phục vụ cho môn học cũng như làm hành trang cho cuộc sống của chúng em sau này.

Chúng em xin chân thành cảm ơn!

Nhóm sinh viên thực hiện đề tài

2 TỔNG QUAN ĐỀ TÀI

- **Lĩnh vực:** Dữ liệu liên quan đến xây dựng và kiểm tra chất lượng vật liệu, tập trung vào phát hiện và phân tích vết nứt trên bề mặt bê tông và mặt đường.
- **Nguồn thu thập dữ liệu:** Các hình ảnh được thu thập từ Đại học Quân đội Nigeria Bui, bang Borno, Nigeria, và được thu thập bởi Omobamije Oluwaseun, một sinh viên ngành kỹ thuật xây dựng, cho dự án tốt nghiệp cuối khóa của mình. Các hình ảnh này được chụp bằng máy bay không người lái DJI Mavic 2 Enterprise (để chụp ở độ cao lớn) và điện thoại thông minh (để chụp ở độ cao thấp hơn chiều cao trung bình của cửa sổ). Bộ dữ liệu được lưu ở định dạng RGB, JPEG và được giảm kích thước xuống còn 227 x 227 pixel. Các hình ảnh này đã được dán nhãn *Negative* (không có vết nứt) và *Positive* (có vết nứt).
- **Mục đích đề tài:**
 - Tìm hiểu và ứng dụng mạng nơ-ron tích chập (CNNs), một công nghệ hiện đại trong lĩnh vực trí tuệ nhân tạo, để giải quyết bài toán thực tế.
 - Nâng cao khả năng phân tích dữ liệu hình ảnh, tiền xử lý dữ liệu và triển khai các thuật toán học sâu.
 - Xây dựng hệ thống tự động nhận diện hình ảnh có vết nứt hay không..
- **Loại dữ liệu:**
 - Dữ liệu không cấu trúc (hình ảnh), cần xử lý để trích xuất các đặc trưng (features) phù hợp cho phân tích và mô hình học sâu CNN.
- **Đọc dữ liệu:**
 - Sử dụng thư viện trong Python Tensorflow/Keras kết hợp OpenCV để đọc hình ảnh.
 - Áp dụng tiền xử lý (resize, normalize).
 - Xây dựng mô hình CNN:
 - * Hình ảnh sẽ được trích xuất các đặc trưng quan trọng thông qua các lớp tích chập và pooling như góc, cạnh đến các chi tiết phức tạp
 - * Các đặc trưng được chuyển qua lớp flatten và fully connected.
 - * Lớp đầu ra sẽ dự đoán nhãn cho ảnh (Hình ảnh xi măng có hay không có vết nứt.)
 - Lưu dữ liệu đã xử lý.

3 CƠ SỞ LÝ THUYẾT

3.1 Giới thiệu

- Mục tiêu của báo cáo: Phân tích hình ảnh bằng công nghệ trí tuệ nhân tạo.
- Vai trò của phân tích hình ảnh trong các lĩnh vực (y tế, tự động hóa, an ninh, v.v.).
- Giới thiệu sơ lược về mạng nơ-ron tích chập (CNN).

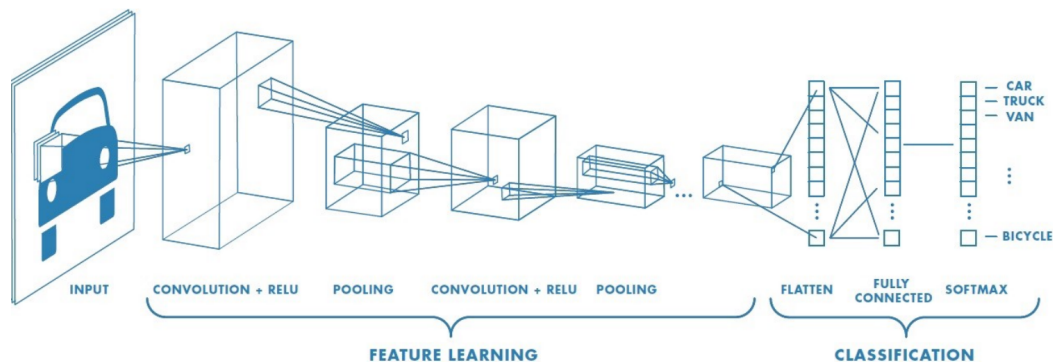
Đặc điểm của dữ liệu:

1. Dữ liệu hình ảnh (phi cấu trúc)
2. Nhãn: Các vết nứt được chú thích cho biết sự xuất hiện, phân loại và mức độ nghiêm trọng (nếu có) của vết nứt. Bao gồm:
 - Nhãn nhị phân: Positive (có nứt) và Negative (không nứt)
 - Phân loại vết nứt: theo chiều dọc, ngang,....
 - Mức độ nghiêm trọng: nông, vừa, sâu,...
3. Siêu dữ liệu (Metadata): Trong một số trường hợp, một số siêu dữ liệu thêm như vị trí địa lí, thời gian ảnh được ghi lại, điều kiện môi trường (nhiệt độ, độ ẩm,...) nên được đính kèm.

3.2 Tổng quan về mạng nơ-ron tích chập (CNN)

3.2.1 CNN là gì?

- Convolutional Neural Network (Mạng nơ-ron tích chập) là mạng nơ-ron nhân tạo giữ vai trò chủ đạo trong các xử lý nhiệm vụ thị giác máy tính khác nhau. Chúng được thiết kế để tự động học và hệ thống các phân cấp không gian của đặc điểm hình ảnh thông qua truyền ngược bằng cách xây dựng các mô hình khối, chẳng hạn như lớp tích chập, lớp tổng hợp và các lớp được kết nối đầy đủ.



Hình 1: Nguyên lý hoạt động của CNN

3.2.2 Lý do CNN phù hợp với phân tích hình ảnh.

- CNN chuyên xử lý dữ liệu dạng lưới như hình ảnh, sử dụng các lớp để tự động phát hiện các mẫu và tính năng. Các thuật toán học máy khác thường yêu cầu trích xuất tính năng thủ công và không hiệu quả đối với các tác vụ liên quan đến hình ảnh.

3.3 Kiến trúc cơ bản của CNN

3.3.1 Các thành phần chính

- Lớp tích chập (Convolution Layer).

Lớp tích chập thực hiện một phép toán học được gọi là tích chập, liên quan đến mối tương quan chéo giữa dữ liệu đầu vào và một tập hợp các hạt nhân có thể học được (bộ lọc) để tạo ra bản đồ tính năng đầu ra. Trọng số hạt nhân ban đầu được ngẫu nhiên và sau đó được học trong quá trình đào tạo mô hình thông qua backpropagation và gradient descent để giảm thiểu chức năng tổn thất.

Các Tham Số Quan Trọng

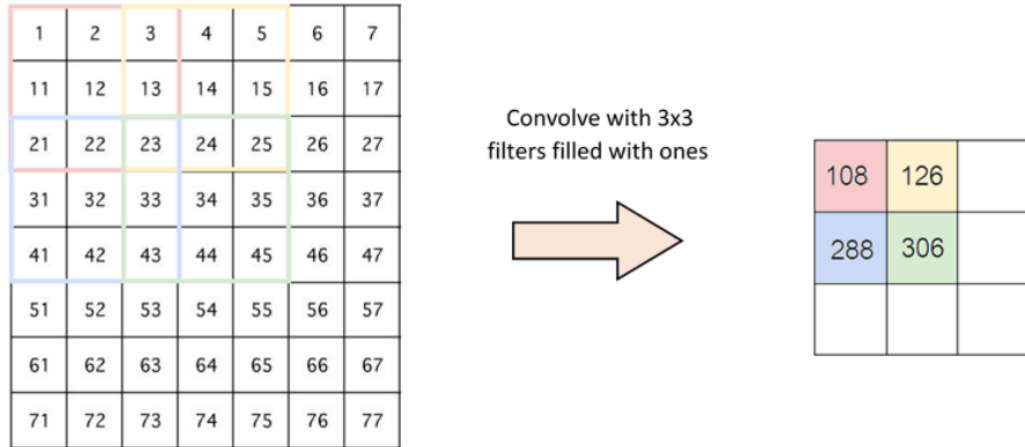
Bộ lọc (Kernel): là một ma trận nhỏ chứa các tham số có thể học được, kích thước cố định, thường là 3x3, 5x5, hoặc 7x7.

Khi thay đổi các tham số của kernel, các đặc trưng được trích xuất từ dữ liệu đầu vào cũng sẽ thay đổi. Điều này dẫn đến việc tạo ra nhiều bản đồ đặc trưng khác nhau từ cùng một ảnh đầu vào. Mỗi kernel sẽ học cách nhận diện các đặc trưng khác nhau, chẳng hạn như các cạnh, góc, hoặc các mẫu phức tạp hơn trong hình ảnh.

Operation	Filter	Convolved Image
Identity	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	
Edge detection	$\begin{bmatrix} 1 & 0 & -1 \\ 0 & 0 & 0 \\ -1 & 0 & 1 \end{bmatrix}$	
	$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$	
	$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$	
Sharpen	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
Box blur (normalized)	$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$	
Gaussian blur (approximation)	$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$	

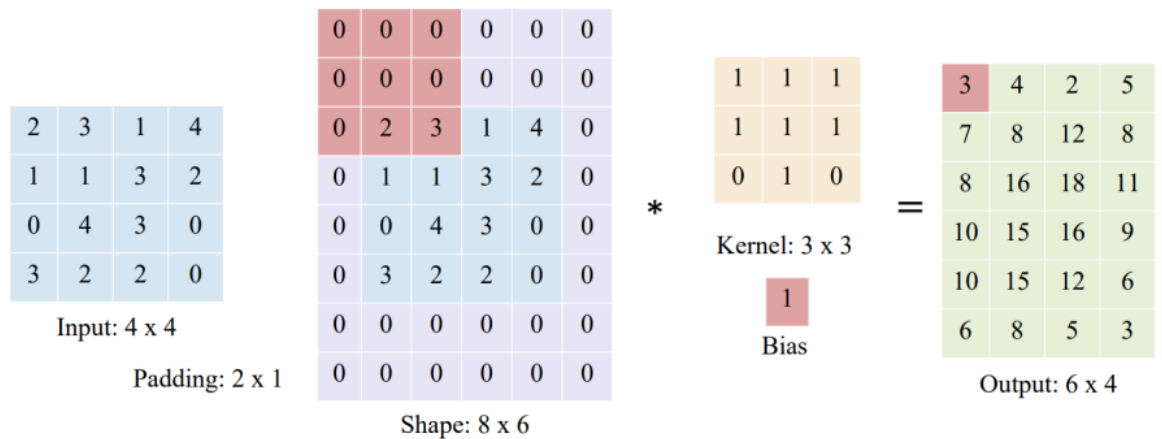
Hình 2: Một số bộ lọc (Kernels) thường gặp

Stride: Số pixel mà bộ lọc di chuyển sau mỗi lần thực hiện phép tích chập. Stride lớn hơn sẽ giảm kích thước của bản đồ đặc trưng.



Hình 3: Hình minh họa bộ lọc 3x3 với stride=1

Padding: Thêm các giá trị 0 xung quanh biên của hình ảnh đầu vào để duy trì kích thước của bản đồ đặc trưng sau khi thực hiện phép tích chập.

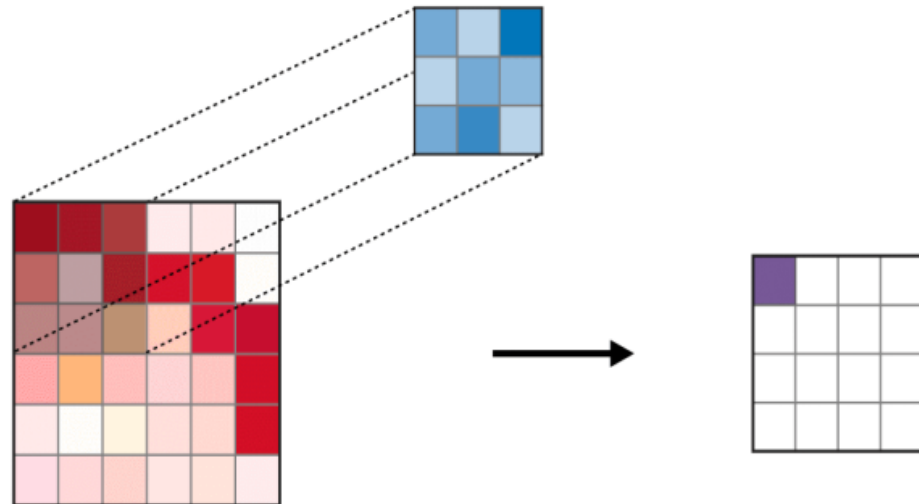


Hình 4: Quá trình tích chập

Quá trình tích chập:

- Đặt Kernel lên góc trên bên trái của hình ảnh đầu vào.
- Kernel di chuyển qua đầu vào theo từng stride. Tại mỗi vị trí, bộ lọc sẽ
 - * Tính tích từng phần tử của mỗi giá trị pixel tương ứng.

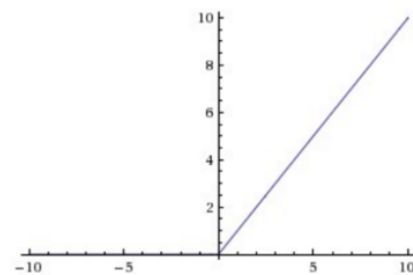
- * Cộng các tích lại để có giá trị pixel ở góc trên bên trái của đầu ra.
- Lặp lại quá trình này được lặp lại cho tất cả các vị trí của bộ lọc trên đầu vào để tạo ra toàn bộ bản đồ đặc trưng.



Hình 5: Lặp lại quá trình để tạo nên bản đồ đặc trưng

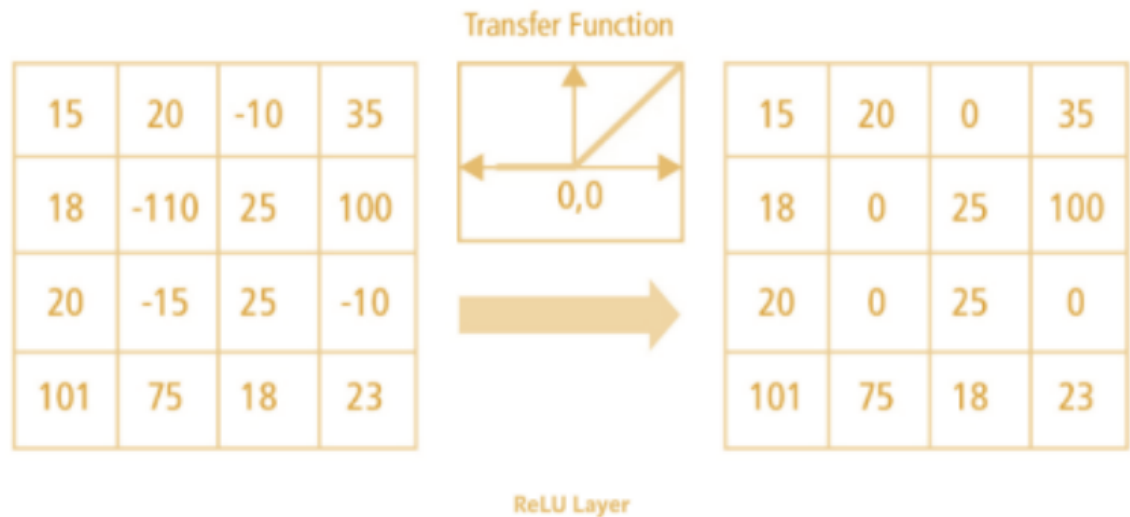
- **Lớp kích hoạt (Activation Layer, thường dùng ReLU):**
 - Là các hàm phi tuyến tính (ví dụ: ReLU) được áp dụng sau khi tích chập để đưa tính phi tuyến vào mô hình, cho phép nó tìm hiểu các mẫu phức tạp.

$$\text{Output} = \text{Max}(\text{zero}, \text{Input})$$



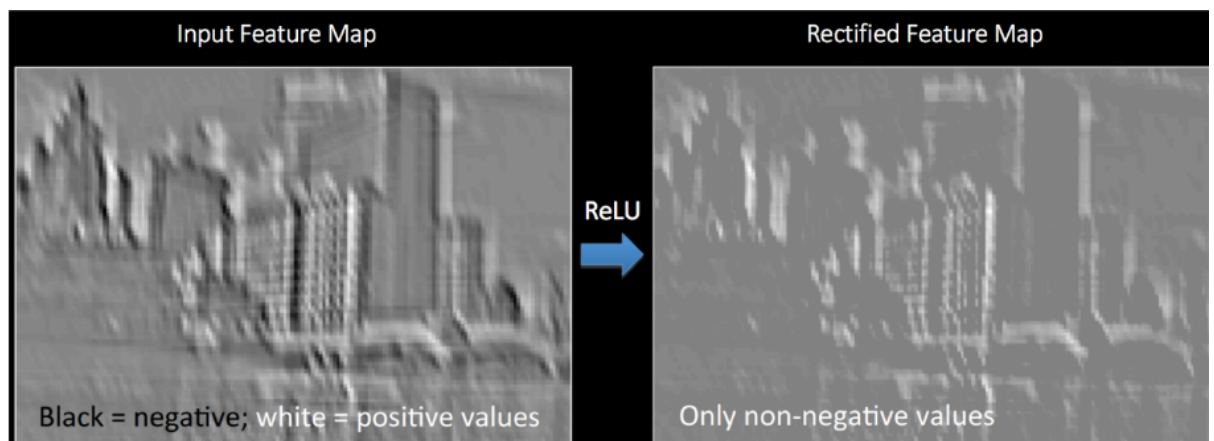
Hình 6: Hàm ReLU

- Hàm ReLU thay thế tất cả các giá trị pixel âm trong bản đồ đặc trưng bằng 0. Điều này giúp loại bỏ các giá trị âm không cần thiết và giữ lại các giá trị dương, giúp mô hình tập trung vào các đặc trưng quan trọng hơn.



Hình 7: Mô tả quá trình hoạt động hàm ReLU

-Ví dụ:



Hình 8: Một ví dụ về hoạt động của hàm ReLU

Phép tích chập là một phép toán tuyến tính, nghĩa là nó chỉ thực hiện các phép nhân và cộng theo từng phần tử của ma trận. Tuy nhiên, hầu hết dữ liệu thực tế mà chúng ta muốn mô hình học đều là phi tuyến tính. Do đó, việc giới thiệu một hàm phi tuyến tính như ReLU giúp mô hình có khả năng học các đặc trưng phi tuyến tính từ dữ liệu. Hàm ReLU có tính chất đơn giản và hiệu quả, chỉ cần thực hiện phép so sánh và thay thế giá trị. Điều này giúp tăng tốc độ tính toán và giảm thời gian huấn luyện mô hình so với các hàm phi

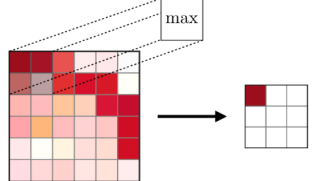
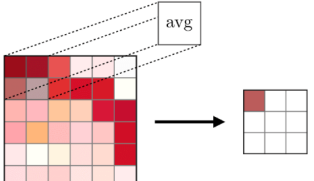
tuyến tính khác như sigmoid hoặc tanh.

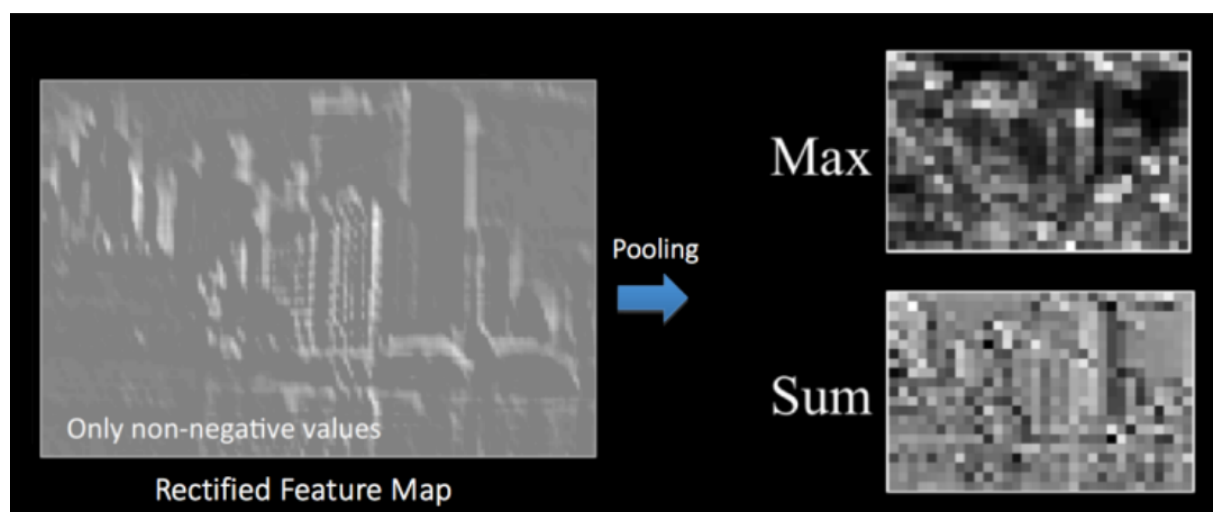
- Lớp gộp (Pooling Layer)

- Là các lớp có chức năng giảm kích thước của dữ liệu đầu vào (input), nhờ đó giảm sự phức tạp của quá trình tính toán và lưu trữ trong khi máy học. Ngoài ra, các lớp gộp còn giúp thu thập các đặc điểm quan trọng của dữ liệu, không làm biến đổi hay mất đi các chi tiết nhỏ trong quá trình lược bớt dữ liệu không liên quan.

-Các kiểu lớp gộp:

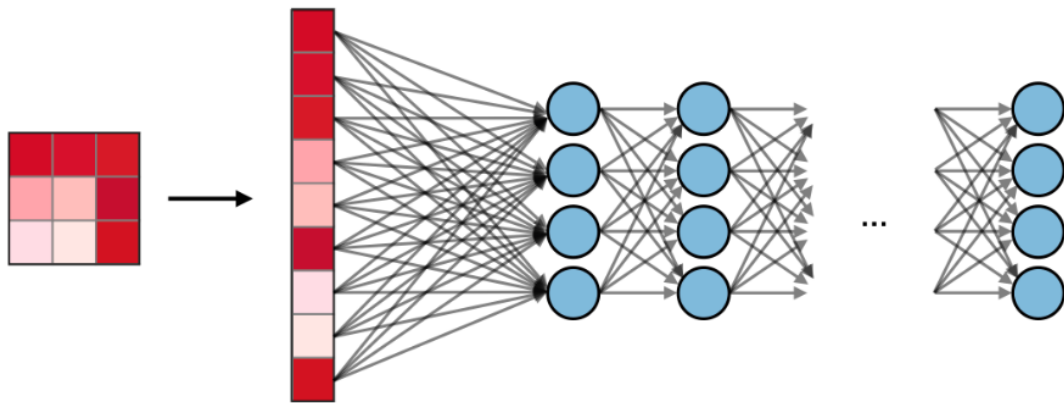
Bảng 1: So sánh Max pooling và Average pooling

Kiểu	Max pooling	Average pooling
Chức năng	Từng phép pooling chọn giá trị lớn nhất trong khu vực mà nó đang được áp dụng	Từng phép pooling tính trung bình các giá trị trong khu vực mà nó đang được áp dụng
Minh họa		
Nhận xét	• Bảo toàn các đặc trưng đã phát hiện • Được sử dụng thường xuyên	• Giảm kích thước feature map • Được sử dụng trong mạng LeNet



Hình 9: Hình ảnh đã qua Max Pooling và Sum Pooling

- Lớp kết nối đầy đủ (Fully Connected Layer)
 - Lớp Fully Connected là lớp mà mỗi neuron trong đó kết nối với tất cả các neuron trong lớp trước. Các kết nối này được đại diện bởi các trọng số (weights) và bậc tự do (biases), mỗi kết nối này có một giá trị số thực được cập nhật trong quá trình huấn luyện. Lớp này giúp mô hình học được các đặc trưng tổng hợp từ các lớp trước đó và đưa ra kết quả cuối cùng (ví dụ: phân loại, hồi quy, ...).
 - VD: Giả sử có n neuron ở lớp trước và m neuron ở lớp hiện tại, thì trọng số (weights) học được sẽ là nm .
 - Chức năng: Phân loại: Các lớp Fully Connected giúp tập hợp các đặc điểm được trích xuất bởi các lớp trước đó (ví dụ: các lớp tích chập) để đưa ra các dự đoán cuối cùng.



Hình 10: Sơ đồ hoạt động lớp Fully Connected

3.3.2 Luồng xử lý dữ liệu trong CNN.

Dữ liệu đi qua nhiều lớp khác nhau, mỗi lớp thực hiện một nhiệm vụ đặc biệt để trích xuất các đặc trưng và học được các mẫu phức tạp từ dữ liệu đầu vào (thường là hình ảnh). Quá trình xử lý này gồm các bước chủ yếu từ việc nhận đầu vào đến khi đưa ra dự đoán cuối cùng.

- Các bước trong xử lý dữ liệu:

- Input Layer: Dữ liệu (hình ảnh) được đưa vào.
- Convolutional Layer: Trích xuất các đặc trưng qua các bộ lọc.
- Activation Function (ReLU): Áp dụng hàm kích hoạt ReLU.
- Pooling Layer: Giảm kích thước của feature maps.
- Flatten Layer: Biến dữ liệu thành vector 1D.
- Fully Connected Layer (Dense): Tổng hợp đặc trưng và đưa ra kết quả phân loại.
- Output Layer: Đưa ra dự đoán cuối cùng (ví dụ: phân loại).



VD: Giả sử bạn có một hình ảnh $28 \times 28 \times 1$ (ảnh grayscale) và muốn phân loại nó thành một trong 10 lớp. Quá trình có thể diễn ra như sau:

- Input: Hình ảnh $28 \times 28 \times 1$.
- Conv Layer: 3×3 filter, output sẽ là $26 \times 26 \times 32$ (32 là số bộ lọc).
- ReLU Activation: Áp dụng ReLU lên $26 \times 26 \times 32$.
- Max Pooling: 2×2 max pooling, output là $13 \times 13 \times 32$.
- Flatten: Biến $13 \times 13 \times 32$ thành vector 1D có chiều dài 5,888.
- FC Layer: Fully connected layer với 64 neuron.
- Output Layer: Dự đoán 10 lớp, với hàm softmax để xác định xác suất cho mỗi lớp.

3.4 Các ưu điểm và hạn chế của CNN

Ưu điểm	Nhược điểm
• Có hiệu quả cao trong xử lý thị giác máy tính: CNN có hiệu suất vượt trội trong xử lý ảnh như nhận biết, phát hiện và phân loại các đặc điểm. • Tự động học có đặc điểm: CNN có khả năng tự học từ các dữ liệu thô thông qua phép tích chập (convolution) Giúp giảm công việc trích xuất thủ công. • Tăng hiệu suất và tính chính xác: Các lớp tích chập (convolutional layers) giúp tăng hiệu suất của mạng nơ-ron nhân tạo (neural network). Thông qua các kỹ thuật như tích chập vi phân (differential convolution) và tích chập nhận dạng nội dung (content-aware convolution) cải thiện sự chính xác của output và giảm chi phí tính toán.	• Yêu cầu nguồn dữ liệu lớn để huấn luyện máy: CNN yêu cầu một bộ dữ liệu lớn để huấn luyện máy hoạt động tốt, điều này có thể gây trở ngại đối với những nghiên cứu/ngành có nguồn dữ liệu khan hiếm. • Tính toán phức tạp: Việc huấn luyện CNN yêu cầu sự tính toán chuyên sâu và tài nguyên phần cứng mạnh, như là CPU có hiệu suất cao. Điều này có thể gây trở ngại cho các tổ chức/ nghiên cứu nhỏ. • Thiếu tính minh bạch: CNN thường được xem là "hộp đen" do cấu trúc phức tạp của nó. Việc hiểu và giải thích các đặc trưng đã học được cùng quá trình ra quyết định của CNN có thể gặp khó khăn, điều này hạn chế tính minh bạch và độ tin cậy của chúng trong các ứng dụng quan trọng, như y tế, tài chính, an ninh,... • Độ nhạy của siêu tham số: CNN rất nhạy với các bộ siêu tham số (Hyperparameters) như kích thước batch, số lượng của lớp,... Quá trình tìm kiếm bộ siêu tham số để tối ưu hoá mô hình có thể mất nhiều thời gian để thử nghiệm và tinh chỉnh.

Bảng 2: So sánh ưu nhược điểm của mô hình CNN

3.5 Nguyên lý hoạt động của CNN trong phân tích hình ảnh

3.5.1 Cách CNN xử lý hình ảnh

- Trích xuất đặc điểm CNN sử dụng nhiều lớp bộ lọc tích chập để trích xuất dữ liệu tự động, chúng sẽ tiến hành quét hình ảnh và nhận diện các đặc điểm như các góc, chất liệu, hình dạng,...
- Phân tích từng lớp trong mô hình CNN:

- Lớp tích chập: trích xuất đặc điểm hình ảnh.
 - Lớp gộp: giảm kích thước ảnh, giữ thông số quan trọng.
 - Kết nối đầy đủ: phân loại hoặc dự đoán kết quả.
- Tự học: CNN sẽ tự động học các đặc điểm từ dữ liệu hình ảnh thô, giúp tối ưu hoá thông qua quá trình truyền ngược (backpropagation) trong khi huấn luyện.

3.5.2 Ví dụ minh họa

Quá trình sử dụng CNN để phân tích hình ảnh siêu âm ổ bụng

1. Thu thập dữ liệu và tiền xử lý dữ liệu

- Bộ dữ liệu: sử dụng ảnh đen trắng được lấy từ 185 nghiên cứu siêu âm bụng lâm sàng, được phân loại thành 11 loại dựa trên chú thích văn bản của các nhà nghiên cứu
- Hình ảnh được cắt, chia tỷ lệ lại thành độ phân giải 256×256 và xáo trộn. Chia bộ dữ liệu thành một bộ đào tạo (4094 hình ảnh từ 136 nghiên cứu) và một bộ thử nghiệm (1423 hình ảnh từ 49 nghiên cứu)

2. Lựa chọn mô hình xử lý

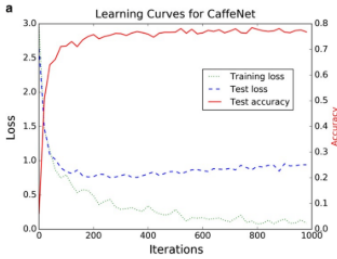
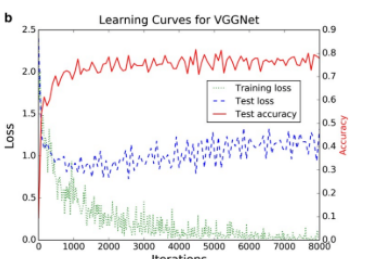
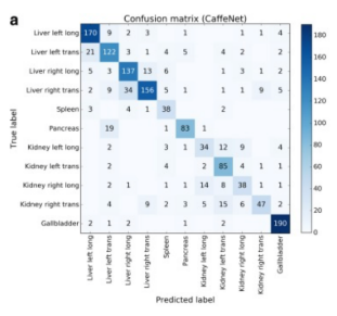
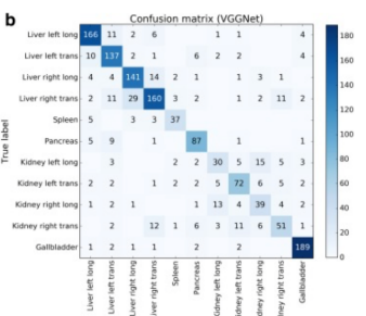
- Mô hình xử lý: Nhóm nghiên cứu lựa chọn sử dụng 2 mô hình đã huấn luyện trước là CaffeNet và VGGNet (2 CNN được thiết kế để thực hiện các tác vụ như phân loại hình ảnh và nhận diện đối tượng).

3. Huấn luyện mô hình

- Trích xuất đặc trưng: Các lớp tích chập của mạng này được cố định để trích xuất đặc trưng. Chỉ các lớp kết nối đầy đủ mới được huấn luyện lại với bộ dữ liệu hình ảnh siêu âm.
- Huấn luyện: Việc huấn luyện lại liên quan đến điều chỉnh trọng số (weight) trong các lớp kết nối đầy đủ để điều chỉnh mạng cho phù hợp với nhiệm vụ phân loại hình ảnh siêu âm bụng.

4. Đánh giá kết quả

- Độ chính xác:
- Mạng CaffeNet: Đạt được độ chính xác 77,3% trong thử nghiệm, với độ chính xác top 2 là 90,4%.
- Mạng VGGNet: Đạt được độ chính xác cao hơn một chút là 77,9%, với độ chính xác top 2 là 89,7%.
- So sánh: Một bác sĩ X-quang đã phân loại chính xác 71,7% hình ảnh thiết lập thử nghiệm, chỉ ra rằng CNN đã vượt trội hơn con người trong nhiệm vụ này.

	CAFFENET	VGGNET
Cấu trúc mạng	- Là phiên bản cải tiến từ mạng AlexNet, có cấu trúc nông hơn. - Số lượng tầng ít với các trọng số nhỏ. - Tập trung xử lý nhanh và chiếm ít bộ nhớ.	- Có cấu trúc sâu hơn, thường gồm các lớp tích chập nhỏ 3x3 - Khả năng học các đặc trưng phức tạp tốt hơn, thường được dùng để xử lý các bài toán phức tạp.
Độ chính xác		 Nhờ cấu trúc sâu hơn, VGGNet có khả năng học các đặc trưng phức tạp và chi tiết hơn, giúp mô hình đạt hiệu suất cao hơn so với Caffenet..
Hiệu suất huấn luyện	Caffenet có kiến trúc nông hơn, giúp tiết kiệm thời gian và tài nguyên tính toán.	VGGNet mất nhiều thời gian huấn luyện hơn, nhưng có khả năng tổng quát tốt hơn trên tập kiểm tra.
Ma trận nhầm lẫn (Confusion Matrix)		 VGGNet cho thấy số lượng phân loại sai (misclassification) ít hơn đáng kể so với CaffeNet, đặc biệt là khi phân biệt các loại ảnh tương tự nhau.

5. Phần code mẫu (Nguồn ChatGPT)

```

1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 from torchvision import datasets, transforms, models
5 from torch.utils.data import DataLoader
6
7 # 1. Data Preparation
8 transform = transforms.Compose([
9     transforms.Grayscale(num_output_channels=3), # Convert grayscale to 3
10     transforms.Resize((256, 256)), # Resize images to 256x256
11     transforms.RandomHorizontalFlip(), # Data augmentation (random
    transforms.HorizontalFlip())

```

```
12     transforms.ToTensor(),                                # Convert images to tensors
13     transforms.Normalize([0.5], [0.5])                  # Normalize data to range
14     [-1, 1])
15
16 # Load training and test datasets
17 train_data = datasets.ImageFolder("path_to_training_data", transform=
18     transform)
19 test_data = datasets.ImageFolder("path_to_test_data", transform=transform)
20
21 train_loader = DataLoader(train_data, batch_size=32, shuffle=True)
22 test_loader = DataLoader(test_data, batch_size=32, shuffle=False)
23
24 # 2. Model Selection
25 caffeNet = models.alexnet(pretrained=True) # Using AlexNet instead of
26     CaffeNet
27 vggnet = models.vgg16(pretrained=True)
28
29 # Freeze convolutional layers
30 for param in caffeNet.features.parameters():
31     param.requires_grad = False
32 for param in vggnet.features.parameters():
33     param.requires_grad = False
34
35 # Modify fully connected layers to fit the classification problem (11 classes
36     )
37 num_classes = 11
38 caffeNet.classifier[6] = nn.Linear(caffeNet.classifier[6].in_features,
39     num_classes)
40 vggnet.classifier[6] = nn.Linear(vggnet.classifier[6].in_features,
41     num_classes)
42
43 # 3. Model Training
44 criterion = nn.CrossEntropyLoss()
45 caffe_optimizer = optim.Adam(caffeNet.classifier.parameters(), lr=0.001)
46 vgg_optimizer = optim.Adam(vggnet.classifier.parameters(), lr=0.001)
47
48 # Training function
49 def train_model(model, optimizer, train_loader, num_epochs=10):
50     model.train()
51     for epoch in range(num_epochs):
52         running_loss = 0.0
53         correct = 0
54         total = 0
55         for images, labels in train_loader:
56             images, labels = images.cuda(), labels.cuda()
57             optimizer.zero_grad()
58             outputs = model(images)
59             loss = criterion(outputs, labels)
60             loss.backward()
61             optimizer.step()
62
63             running_loss += loss.item()
64             _, predicted = outputs.max(1)
65             total += labels.size(0)
66             correct += predicted.eq(labels).sum().item()
67
68         print(f"Epoch [{epoch + 1}/{num_epochs}], Loss: {running_loss/len(
69             train_loader):.4f}, Accuracy: {100 * correct / total:.2f}%")
70
71 # Train CaffeNet model
72 caffeNet = caffeNet.cuda()
```

```
67 print("Training CaffeNet...")
68 train_model(caffenet, caffe_optimizer, train_loader)
69
70 # Train VGGNet model
71 vggnet = vggnet.cuda()
72 print("Training VGGNet...")
73 train_model(vggnet, vgg_optimizer, train_loader)
74
75 # 4. Model Evaluation
76 def evaluate_model(model, test_loader):
77     model.eval()
78     correct = 0
79     total = 0
80     with torch.no_grad():
81         for images, labels in test_loader:
82             images, labels = images.cuda(), labels.cuda()
83             outputs = model(images)
84             _, predicted = outputs.max(1)
85             total += labels.size(0)
86             correct += predicted.eq(labels).sum().item()
87
88     accuracy = 100 * correct / total
89     print(f"Test accuracy: {accuracy:.2f}%")
90     return accuracy
91
92 # Evaluate CaffeNet model
93 print("Evaluating CaffeNet...")
94 caffe_net_accuracy = evaluate_model(caffenet, test_loader)
95
96 # Evaluate VGGNet model
97 print("Evaluating VGGNet...")
98 vggnet_accuracy = evaluate_model(vggnet, test_loader)
99
100 # Print final results
101 print(f"CaffeNet accuracy: {caffe_net_accuracy:.2f}%")
102 print(f"VGGNet accuracy: {vggnet_accuracy:.2f}%")
```

6. Giải thích code

1. Chuẩn bị dữ liệu:

- Chuyển đổi ảnh xám: Vì ảnh siêu âm là grayscale, cần chuyển đổi thành ảnh có 3 kênh để phù hợp với mô hình tiền huấn luyện.
- Tăng cường dữ liệu: Sử dụng các kỹ thuật như lật ảnh ngang để cải thiện tính đa dạng của dữ liệu.

2. Lựa chọn mô hình:

- CaffeNet được mô phỏng bằng AlexNet vì CaffeNet không có sẵn trong thư viện PyTorch.
- VGGNet sử dụng VGG-16 (16 lớp).

3. Huấn luyện:

- Đóng băng các lớp convolutional: Giữ nguyên các trọng số của các lớp trích xuất đặc trưng.
- Chỉ huấn luyện lại các lớp fully connected.

4. Đánh giá:

- Kiểm tra độ chính xác trên tập dữ liệu kiểm tra để đo hiệu năng của mô hình.

3.5.3 Các ứng dụng thực tế của CNN

- Thị giác máy tính (vd: nhận diện gương mặt, phân loại hình ảnh)
 - Ứng dụng: Tính năng mở khoá bằng gương mặt Face ID trên Iphone sử dụng CNN để nhận diện người dùng; Google Photos sử dụng CNN để tự động nhận diện và sắp xếp ảnh theo đối tượng (người, cảnh vật, động vật).
- Chẩn đoán y tế qua ảnh (vd: chụp X-quang, MRI): CNN được sử dụng rộng rãi trong các nhiệm vụ như phân đoạn, phát hiện bất thường và phân loại bệnh trong hình ảnh y tế. Chẩn đoán bệnh võng mạc tiểu đường từ ảnh đáy mắt (Google AI đã phát triển giải pháp này).
- Xe tự hành (nhận diện đường, vật cản): CNN được sử dụng trong hệ thống của Tesla để giúp xe tự động lái trong các điều kiện đường xá phức tạp.
- Thương mại điện tử: Shopee và Taobao sử dụng tính năng này để gợi ý sản phẩm tương tự theo lựa chọn của người dùng.
- Nhận dạng ký tự quang học (OCR): Google Translate sử dụng CNN để nhận diện và dịch văn bản từ hình ảnh chụp.
- Giải trí: Ứng dụng: Các công cụ AI như Remini sử dụng CNN để phục hồi ảnh cũ thành ảnh sắc nét. Các video Deepfake thường sử dụng CNN để tái tạo khuôn mặt với độ chân thực cao.
- Phân tích video và giám sát an ninh: Một hệ thống giám sát có thể sử dụng CNN để nhận diện một người rút vũ khí và LSTM (Long Short-Term Memory) để phân tích động thái tiếp theo của người này, như liệu họ có đang di chuyển theo hướng nguy hiểm hay không.

3.5.4 Thách thức và hướng phát triển

1. Thách Thức

- Dữ liệu thiếu chất lượng hoặc mất cân bằng: Với dữ liệu đào tạo hạn chế, CNN có thể “quá hoàn hảo” (overfitting), dẫn đến gây nhiễu thay thế vì các giá trị có ý nghĩa. Các kỹ thuật như dropout và tăng cường dữ liệu được sử dụng để giảm thiểu điều này.

- Giới thiệu về Dropout

+ Dropout là một kỹ thuật regularization, sử dụng để giảm hiện tượng overfitting trong mạng nơ-ron nhân tạo. Trong quá trình huấn luyện, nó tạm thời loại bỏ (drop) ngẫu nhiên một số neuron cùng với kết nối của chúng. Điều này buộc mạng phải học cách dựa vào nhiều đặc trưng hơn thay vì chỉ một số ít đặc trưng cụ thể, từ đó cải thiện khả năng khái quát hóa (generalization), giúp mô hình chính xác các biểu diễn đa dạng và giảm nguy cơ bị lệ thuộc quá mức vào một số đặc trưng cụ thể.

- Giới thiệu về Tăng cường dữ liệu

+ Tăng cường dữ liệu là một kỹ thuật đặc biệt phổ biến trong Deep Learning, nhằm tăng kích thước và đa dạng của tập dữ liệu huấn luyện bằng cách áp dụng các biến đổi ngẫu nhiên như lật, phóng to/nhỏ, cắt, quay,... lên dữ liệu hiện có mà không làm thay đổi nhãn hoặc đặc tính quan trọng, ngăn chặn mạng học các đặc trưng không quan trọng hoặc nhiễu.

- Thời gian và chi phí tính toán cao: Việc huấn luyện mô hình CNN yêu cầu chi phí và nguồn dữ liệu lớn để đào tạo sâu.

2. Hướng phát triển

- Kết hợp CNN với các mô hình khác (RNN, Transformer)
 - Ví dụ: Mô hình kết hợp CNN-SVM: Tích hợp CNN với Máy hỗ trợ vector (Support Vectors Machine) để nhận dạng chữ số viết tay, nhằm kết hợp khả năng trích xuất tính năng của CNN với sức mạnh phân loại của SVM. Mô hình lai này đã đạt được tỷ lệ nhận dạng 99,81% trên tập dữ liệu MNIST (Modified National Institute of Standards and Technology), vượt trội hơn các nghiên cứu khác trên cùng một cơ sở dữ liệu.
- Mạng CNN nhẹ hơn để dùng trên thiết bị di động:
 - Tối ưu hóa các mô hình học sâu (deep learning) sao cho chúng có thể hoạt động hiệu quả trên các thiết bị có tài nguyên hạn chế về bộ nhớ, tốc độ xử lý và tiêu thụ điện năng.
 - Ví dụ: MobileNet là một trong những mô hình CNN tối ưu dành cho các thiết bị di động. Nó sử dụng một kỹ thuật gọi là sự tích chập có thể tách rời theo chiều sâu (depthwise separable convolutions), giúp giảm đáng kể số lượng tham số và phép toán so với các mạng CNN truyền thống. MobileNet có thể đạt được hiệu suất cao trong các tác vụ nhận diện hình ảnh trong khi vẫn giữ được kích thước nhỏ gọn, rất phù hợp cho các thiết bị di động.

3.5.5 Kết luận

Tổng kết vai trò của CNN trong phân tích hình ảnh

- Tự động trích xuất đặc trưng: CNN giúp nhận diện các đặc trưng của hình ảnh mà không cần sự can thiệp thủ công.
- Độ chính xác cao: CNN đạt hiệu quả tốt trong các bài toán nhận diện hình ảnh, đặc biệt trong y tế, an ninh, và tự động hóa.
- Ứng dụng rộng rãi: CNN được ứng dụng trong nhận dạng hình ảnh y tế, video giám sát, giao thông thông minh, và robot. Tiềm năng và ứng dụng rộng rãi trong tương lai.
- Y tế: Giúp chẩn đoán bệnh qua hình ảnh y tế, như X-quang, MRI.
- An ninh: Phân tích video và nhận diện hành vi trong hệ thống giám sát.
- Tự động hóa và robot: Hỗ trợ nhận diện và tương tác với môi trường trong các ngành công nghiệp.
- Edge AI: Mô hình CNN nhẹ cho điện thoại và thiết bị di động, giúp tiết kiệm băng thông và tài nguyên.
- Giao thông thông minh và ô tô tự lái: Sử dụng CNN để nhận diện biển báo và phương tiện trong môi trường giao thông.

4 ĐOẠN CODE KAGGLE

4.1 Nhập các thư viện cần thiết

```
1 !pip install visualkeras
2 import os
3 import warnings
4 import itertools
5 import cv2
6 import seaborn as sns
7 import pandas as pd
8 import numpy as np
9 from PIL import Image
10 from sklearn.utils import class_weight
11 from sklearn.metrics import confusion_matrix, classification_report
12 from collections import Counter
13
14 import tensorflow as tf
15 import tensorflow_addons as tfa
16 import visualkeras
17 import plotly.express as px
18 import matplotlib.pyplot as plt
19 from sklearn.metrics import multilabel_confusion_matrix
20
21 from tensorflow.keras.preprocessing.image import load_img
22 from tensorflow.keras.utils import plot_model
23 from tensorflow.keras import layers
24 from tensorflow.keras import regularizers
25 from sklearn.model_selection import train_test_split
26 from keras.preprocessing.image import ImageDataGenerator
27
28 warnings.filterwarnings('ignore')
29 %matplotlib inline
```

4.2 Thiết lập các thông số chung

```
1 # General parameters
2 epochs = 15
3 image_size = 240
4 np.random.seed(42)
5 tf.random.set_seed(42)
```

- epochs = 15: trải qua 15 lần huấn luyện
- image_size = 240: các hình ảnh đầu vào sẽ được thay đổi kích thước thành 240x240 pixels.
- Seed là giá trị khởi tạo. Có giá trị này sẽ tính được các số ngẫu nhiên tiếp theo.
- Seed cho NumPy và TensorFlow giúp đảm bảo tính lặp lại của các phép toán ngẫu nhiên, rất quan trọng cho việc tái tạo kết quả thí nghiệm và mô hình học máy.

4.3 Tải dữ liệu, tiền xử lý và trực quan hóa

```
1 os.listdir("/kaggle/input/concrete-and-pavement-crack-images",)
```

os.listdir() là một hàm trong mô-đun os dùng để liệt kê tất cả các tệp và thư mục trong thư mục chỉ định.



```
1 folder_path = ("/kaggle/input/concrete-and-pavement-crack-images")
```

Gán đường dẫn vào biến `folder_path`, giúp bạn dễ dàng sử dụng lại đường dẫn này ở nhiều nơi trong mã của bạn mà không cần phải lặp lại chuỗi đường dẫn dài.

```
1 import os
2 import cv2
3 import numpy as np
4
5 def load_and_process_dataset(folder_path):
6     """Loads and processes images using alternative while loop structures."""
7
8     dataset = []
9     labels = []
10    class_folders = ['Negative', 'Positive']
```

- load_and_process_dataset(folder_path): Hàm này sẽ tải và xử lý ảnh, sử dụng cấu trúc vòng lặp while thay vì các kiểu vòng lặp thông thường khác (như for)

. - dataset: Là một danh sách sẽ lưu trữ các ảnh sau khi được xử lý (mỗi ảnh là một mảng NumPy).

- labels: Là một danh sách lưu trữ các nhãn của ảnh (ví dụ, 0 cho lớp "Negative" và 1 cho lớp "Positive").

- class_folders: Là danh sách chứa tên của các thư mục con ("Negative" và "Positive"). Mỗi thư mục này chứa các ảnh thuộc về hai lớp phân loại tương ứng.

```
1 class_index = 0
2 while class_index < len(class_folders):
3     class_folder = class_folders[class_index]
4     images_path = os.path.join(folder_path, class_folder)
```

- Khởi tạo và Vòng lặp ngoài (vòng lặp qua các lớp):

- class_index = 0: Khởi tạo chỉ số của lớp (class).
- while class_index < len(class_folders): Vòng lặp chạy qua tất cả các thư mục trong class_folders. Mỗi thư mục đại diện cho một lớp khác nhau.
- class_folder = class_folders[class_index]: Lấy tên thư mục hiện tại đại diện cho một lớp.
- images_path = os.path.join(folder_path, class_folder): Xây dựng đường dẫn đầy đủ tới thư mục chứa hình ảnh của lớp.

```
1 image_index = 0
2 while True: # Loop infinitely until a "break" occurs
```

- image_index = 0: Khởi tạo chỉ số cho hình ảnh.

- while True: Vòng lặp vô hạn, sẽ dừng lại khi có lệnh break (trong trường hợp này, khi hết hình ảnh trong thư mục).

```
1         try:
2             image_name = os.listdir(images_path)[image_index] # Access by
3             index
4             image_path = os.path.join(images_path, image_name)
5
6             image = cv2.imread(image_path)
7             image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB) # Convert to RGB
8             image = cv2.resize(image, (240, 240))
9
10            dataset.append(image)
11            labels.append(class_index) # Use class_index directly for labels
12
13            image_index += 1
14        except IndexError: # Handle end of image list
15            break # Exit the inner loop
16    class_index += 1
```

- `os.listdir(images_path)[image_index]`: Lấy tên của hình ảnh tại vị trí `image_index` trong thư mục `images_path`.
- `image_path = os.path.join(images_path, image_name)`: Xây dựng đường dẫn đầy đủ tới hình ảnh.
- `cv2.imread(image_path)`: Đọc hình ảnh từ đường dẫn.
- `cv2.cvtColor(image, cv2.COLOR_BGR2RGB)`: Chuyển đổi hình ảnh từ không gian màu BGR (được sử dụng trong OpenCV) sang RGB (phổ biến trong các thư viện học máy).
- `cv2.resize(image, (240, 240))`: Thay đổi kích thước hình ảnh về 240x240 pixel.
- `dataset.append(image)`: Thêm hình ảnh đã xử lý vào danh sách `dataset`.
- `labels.append(class_index)`: Thêm nhãn lớp vào danh sách `labels`. Nhãn này tương ứng với chỉ số của lớp hiện tại (`class_index`).
- `image_index += 1`: Tăng chỉ số hình ảnh để xử lý hình ảnh tiếp theo.
- `except IndexError`: Xử lý khi hết hình ảnh trong thư mục (lỗi `IndexError` xảy ra khi chỉ số hình ảnh vượt quá số lượng hình ảnh có trong thư mục). Khi đó, vòng lặp dừng lại.
- `class_index += 1`: Sau khi tất cả hình ảnh trong một lớp được xử lý, chỉ số lớp (`class_index`) tăng lên để xử lý lớp tiếp theo.

```
1     return np.array(dataset), np.array(labels)
```

Sau khi xử lý xong tất cả các lớp và hình ảnh, hàm trả về hai mảng NumPy: `dataset` chứa tất cả hình ảnh đã xử lý và `labels` chứa các nhãn lớp tương ứng.

```
1     # Assuming 'folder_path' is already a string
2     dataset, labels = load_and_process_dataset(folder_path)
3
4     # Convert to NumPy arrays
5     dataset = np.array(dataset)
6     lab = np.array(labels)
7
8     # Print shapes
9     print(dataset.shape, labels.shape)
```

- Gọi hàm `load_and_process_dataset(folder_path)` để tải và xử lý dữ liệu từ thư mục gốc `folder_path`.
- Chuyển dataset và labels thành mảng NumPy (mặc dù trong hàm đã trả về dưới dạng NumPy).
- In ra kích thước của bộ dữ liệu và nhãn (số lượng hình ảnh và số lượng lớp).

(30000, 240, 240, 3) (30000,)

30000: Tổng số lượng hình ảnh trong bộ dữ liệu.
240, 240: Mỗi hình ảnh có kích thước 240x240 pixel.
3: Mỗi pixel có 3 kênh màu (RGB: đỏ, xanh lá, xanh dương).

```
1 x_train, x_test, y_train, y_test = train_test_split(dataset, lab, test_size  
=0.2, shuffle=True, random_state=42)
```

- `-train_test_split`: Là một hàm trong thư viện `sklearn.model_selection`, dùng để chia một tập dữ liệu lớn thành 2 phần: tập huấn luyện (train) và tập kiểm tra (test).
- `test_size=0.2`: Quy định tỷ lệ dữ liệu được dùng làm tập kiểm tra. Ở đây, 0.2 có nghĩa là 20% dữ liệu sẽ được tách ra làm tập kiểm tra, và 80% còn lại sẽ làm tập huấn luyện.
- `shuffle=True`: Quy định rằng dữ liệu sẽ được xáo trộn ngẫu nhiên trước khi chia. Điều này giúp đảm bảo rằng dữ liệu huấn luyện và kiểm tra được phân bố đồng đều hơn, tránh hiện tượng dữ liệu bị chia không đồng đều.
- `random_state=42`: Một giá trị cố định dùng để đặt seed cho bộ tạo số ngẫu nhiên. Điều này đảm bảo rằng khi chạy lại cùng một mã lệnh, kết quả chia dữ liệu sẽ luôn giống nhau.

```
1 import matplotlib.pyplot as plt  
2  
3 def plot_state(state):  
4     images = [load_img(os.path.join(folder_path, state, img_name), target_size=(  
image_size, image_size))  
5                 for img_name in os.listdir(os.path.join(folder_path, state))[:9]]  
6  
7     fig, axes = plt.subplots(3, 3, figsize=(12, 12))  
8     for ax, img in zip(axes.flat, images):  
9         ax.imshow(img)  
10    plt.show()
```

- Tham số `state`: Đại diện cho một trạng thái cụ thể (tên thư mục con) chứa các hình ảnh liên quan. Ví dụ: Nếu `state = 'cats'`, thì hàm sẽ lấy hình ảnh từ thư mục có tên 'cats'.
- `images` (Danh sách chứa hình ảnh):
 - `os.path.join(folder_path, state, img_name)`: Tạo đường dẫn đầy đủ đến mỗi tệp hình ảnh trong thư mục con của trạng thái.

- `os.listdir(os.path.join(folder_path, state))[:9]`: Lấy danh sách tối đa 9 tệp đầu tiên trong thư mục state.
- `load_img(...)`: Tải và chỉnh kích thước của mỗi ảnh
- `target_size=(image_size, image_size)`: Quy định kích thước ảnh đầu ra.
Kết quả: Một danh sách chứa 9 hình ảnh đã được tải và chỉnh kích thước.

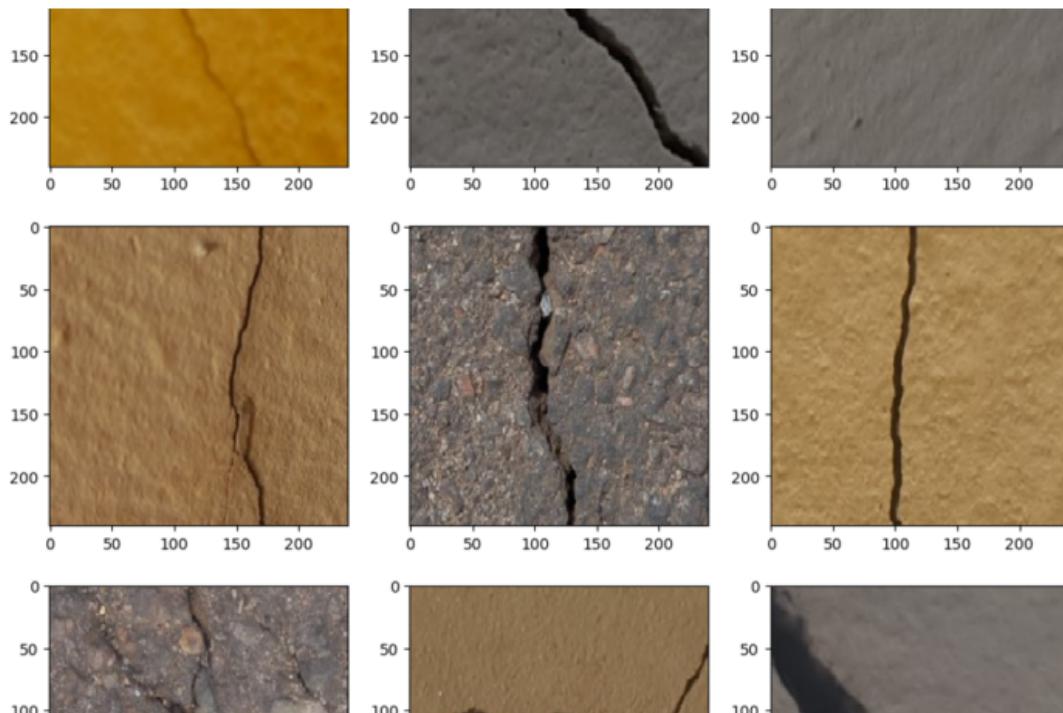
Tạo lưới hình ảnh (Grid layout):

- `fig, axes = plt.subplots(3, 3, figsize=(12, 12))`: Tạo một lưới 3x3 để hiển thị các ảnh. Kích thước của toàn bộ hình là 12x12 inch.
- `for ax, img in zip(axes.flat, images):`: Duyệt qua từng ô trên lưới (ax) và từng ảnh trong danh sách images
- `ax.imshow(img)`: Hiển thị hình ảnh img trong ô ax.
- `plt.show()`: Hiển thị lưới hình ảnh sau khi đã sắp xếp.
- `ax.axis('off')`: Tắt các trục x và y để hình ảnh hiển thị rõ hơn mà không có các nhãn số.

```
1 plot_state('Positive')
```

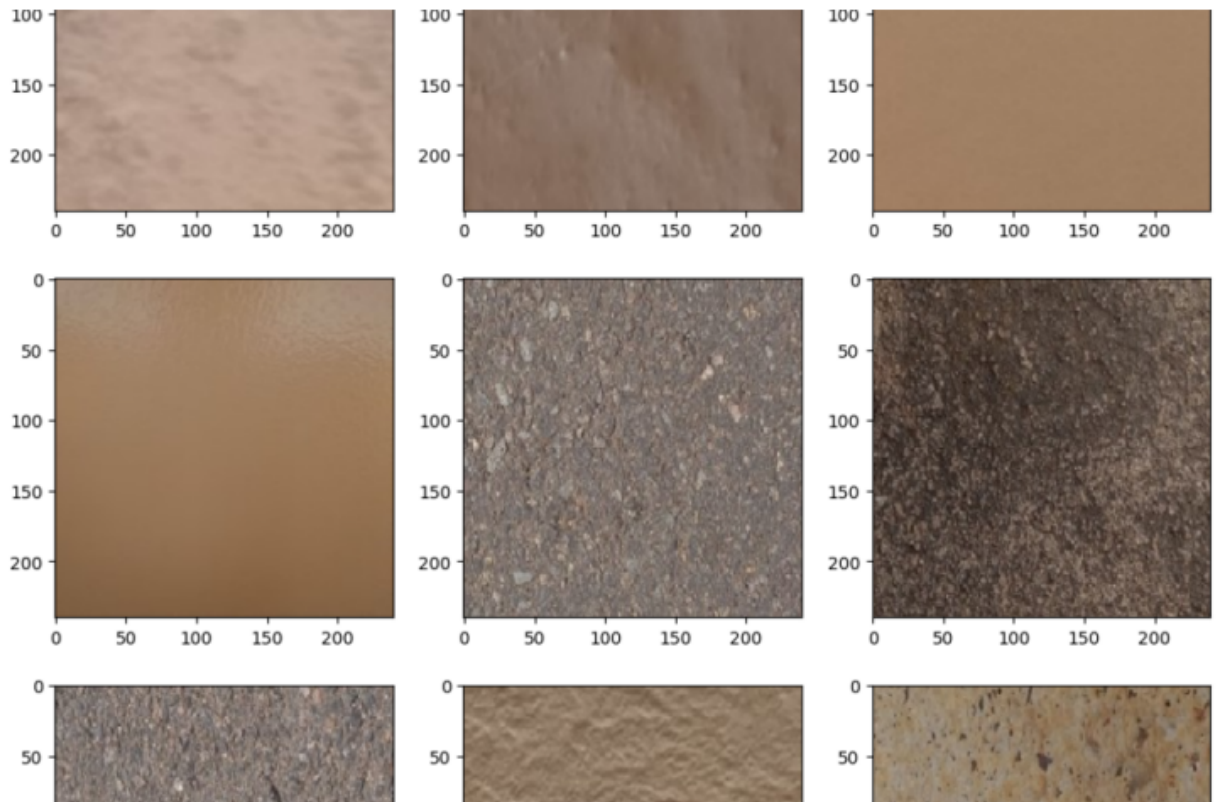
Hàm sẽ thực hiện:

- Tìm thư mục con có tên 'Positive' trong đường dẫn folder_path.
- Lấy tối đa 9 hình ảnh đầu tiên trong thư mục đó. (Không lấy ngẫu nhiên)
- Hiển thị chúng trong một lưới 3x3 với kích thước mỗi ô cân đối.



Tương tự cho Negative:

```
1 plot_state("Negative")
```



4.4 Lập mô hình sử dụng CNN

```
1 model = tf.keras.Sequential([
2
3     tf.keras.layers.Conv2D(filters=32, kernel_size=(3,3), strides=(2,2), activation=
4         "relu", padding="valid", input_shape=(pic_size, pic_size, 3)),
5     tf.keras.layers.MaxPooling2D((2, 2)),
6     tf.keras.layers.Conv2D(filters=32, kernel_size=(3,3), strides=(2,2), activation=
7         "relu", padding="valid"),
8     tf.keras.layers.MaxPooling2D((2, 2)),
9
10    tf.keras.layers.Flatten(),
11    tf.keras.layers.Dense(units=64, activation='relu',
12        kernel_regularizer=regularizers.L1L2(l1=1e-3, l2=1e-3),
13        bias_regularizer=regularizers.L2(1e-2),
14        activity_regularizer=regularizers.L2(1e-3)),
15    tf.keras.layers.Dropout(0.5),
16    tf.keras.layers.Dense(units=1, activation='sigmoid'),
17 ])
```

1. `tf.keras.Sequential(...)`

- Ý nghĩa: Đây là cách tạo một mô hình tuần tự (sequential model), trong đó các tầng (layers) được xếp chồng tuần tự từ đầu vào đến đầu ra.
- Cách dùng: Dùng khi các tầng được tổ chức theo thứ tự tuyến tính, không có nhánh rẽ hay kết nối phức tạp.

2. `tf.keras.layers.Conv2D`

Đây là lớp tích chập 2D, thường dùng để xử lý dữ liệu đầu vào dạng hình ảnh (2D). Lớp này sẽ áp dụng các bộ lọc (filters) lên hình ảnh để phát hiện các đặc trưng (ví dụ: cạnh, góc, kết cấu).

3. `filters=32`

- Số lượng bộ lọc: Ở đây, lớp sẽ sử dụng 32 bộ lọc.
- Mỗi bộ lọc sẽ quét qua ảnh đầu vào để tạo ra 32 bản đồ đặc trưng (feature maps).
- Nhiều bộ lọc giúp học được nhiều đặc trưng khác nhau từ dữ liệu.

4. `kernel_size=(3, 3)`

- Kích thước của bộ lọc (hay kernel). Trong trường hợp này, kích thước của mỗi bộ lọc là 3x3 pixel.
- Bộ lọc này sẽ trượt qua hình ảnh và thực hiện phép nhân tích chập tại từng vị trí.

5. `strides=(2, 2)`

- Bước di chuyển của bộ lọc:
 - + Khi bộ lọc quét qua hình ảnh, nó sẽ di chuyển 2 pixel mỗi lần theo cả chiều ngang và dọc.
 - + Điều này làm giảm kích thước đầu ra (tức là giảm số lượng pixel trong bản đồ đặc trưng) Stride là số lượng pixel mà bộ lọc di chuyển mỗi lần khi quét qua dữ liệu đầu vào.
- Ví dụ:
 - + `stride=(1, 1)`: Bộ lọc di chuyển 1 pixel một lần (rất chi tiết, giữ nhiều thông tin).
 - + `stride=(2, 2)`: Bộ lọc di chuyển 2 pixel một lần (nhanh hơn, giảm độ chi tiết).

6. `activation="relu"`

- Hàm kích hoạt: ReLU (Rectified Linear Unit) được sử dụng.
- Nó thay thế tất cả các giá trị âm trong đầu ra của tích chập bằng 0.
- Điều này giúp tăng tính phi tuyến và cải thiện khả năng học của mạng.

7. `padding="valid"`

- Không thêm điểm "đệm" xung quanh biên ảnh, làm kích thước đầu ra nhỏ hơn đầu vào.

8. `input_shape=(pic_size, pic_size, 3)` - Đầu vào là ảnh có kích thước `(pic_size, pic_size)` và 3 kênh màu (RGB).

4.4.1 Chi tiết các lớp trong mô hình CNN

1. Lớp gộp tối đa đầu tiên (MaxPooling2D)

```
1 tf.keras.layers.MaxPooling2D((2, 2)),
```

- `pool_size=(2,2)`: Áp dụng gộp tối đa (max pooling) với kích thước 2x2.

Chức năng

- Giảm kích thước không gian của ảnh, giữ lại các đặc trưng quan trọng nhất.
- Giúp giảm độ phức tạp của mô hình và tăng tính bền vững với dịch chuyển (translation)

2. Lớp tích chập thứ hai (Conv2D)

```
1 tf.keras.layers.Conv2D(filters=32, kernel_size=(3,3), strides=(2,2),  
activation="relu", padding="valid"),
```

Tương tự lớp tích chập đầu tiên, nhưng không cần xác định `input_shape` vì nó được tự động suy diễn từ lớp trước đó.

3. Lớp gộp tối đa thứ hai (MaxPooling2D)

```
1 tf.keras.layers.MaxPooling2D((2, 2)),
```

Tương tự lớp gộp đầu tiên, giúp tiếp tục giảm kích thước không gian của ảnh.

4. Lớp dàn phẳng (Flatten)

```
1 tf.keras.layers.Flatten(),
```

Chuyển tensor 3D (chiều rộng, chiều cao, số lượng kênh) thành vector 1D để chuẩn bị làm đầu vào cho lớp Dense.

5. Lớp Dense đầu tiên (Fully Connected Layer)

```
1 tf.keras.layers.Dense(units=64, activation='relu',  
2 kernel_regularizer=regularizers.L1L2(l1=1e-3, l2=1e-3),  
3 bias_regularizer=regularizers.L2(1e-2),  
4 activity_regularizer=regularizers.L2(1e-3)),
```

- `units=64`: Lớp này có 64 nơ-ron.
- **%Regularization** trong Machine Learning là một kỹ thuật được sử dụng rộng rãi để **ngăn chặn hiện tượng overfitting**. Overfitting xảy ra khi mô hình học quá kỹ các dữ liệu huấn luyện, đến mức nó bắt đầu **"học" cả những nhiễu và đặc điểm ngẫu nhiên trong dữ liệu**, dẫn đến hiệu suất kém trên dữ liệu kiểm thử.
- Regularization giúp mô hình đơn giản hơn, tập trung vào các đặc trưng quan trọng, từ đó nâng cao khả năng dự đoán trên dữ liệu mới.

```
1 kernel_regularizer=regularizers.L1L2(l1=1e-3, l2=1e-3),
```

Áp dụng regularization L1 và L2 để giảm overfitting.

```
1 bias_regularizer=regularizers.L2(1e-2),
```

Áp dụng regularization L2 cho các bias (độ lệch).

```
1 activity_regularizer=regularizers.L2(1e-3)),
```

Áp dụng regularization L2 cho các đầu ra (output activity).

6. Lớp Dropout

```
1 tf.keras.layers.Dropout(0.5),
```

Ngẫu nhiên bỏ qua 50% nơ-ron trong quá trình huấn luyện để giảm overfitting.

7. Lớp Dense cuối cùng (Output Layer)

```
1 tf.keras.layers.Dense(units=1, activation='sigmoid'),
```

- units=1: Lớp này có 1 nơ-ron, phù hợp cho phân loại nhị phân (binary classification).
- activation='sigmoid': Sử dụng hàm kích hoạt Sigmoid, trả về giá trị xác suất trong khoảng (0, 1).

4.4.2 Tóm tắt hoạt động của mô hình

1. Input

- Nhận ảnh đầu vào kích thước (pic_size, pic_size, 3).

2. Feature Extraction

- Hai lớp tích chập (Conv2D) kết hợp với hai lớp gộp tối đa (MaxPooling2D) giúp trích xuất các đặc trưng quan trọng.

3. Flattening

- Chuyển các đặc trưng trích xuất thành vector 1D.

4. Classification

- Lớp Dense (64 nơ-ron) với regularization giúp học sâu hơn các đặc trưng.
- Dropout giúp giảm overfitting.
- Lớp đầu ra (1 nơ-ron) sử dụng hàm sigmoid để dự đoán xác suất thuộc một trong hai lớp.

**Biên dịch mô hình(model.compile)*

-optimizer='adam': sử dụng thuật toán tối ưu hóa Adam để huấn luyện mô hình. Adam tự động điều chỉnh tốc độ học (learning rate) cho từng tham số, giúp mô hình hội tụ nhanh hơn và hiệu quả hơn.

-loss='binary_crossentropy': Hàm mất mát (loss function) dùng cho bài toán phân loại nhị phân.

-metrics=['accuracy']: Đánh giá mô hình dựa trên tỷ lệ dự đoán đúng.

$$Accuracy = \frac{\text{Số dự đoán đúng}}{\text{Tổng số dự đoán}}$$

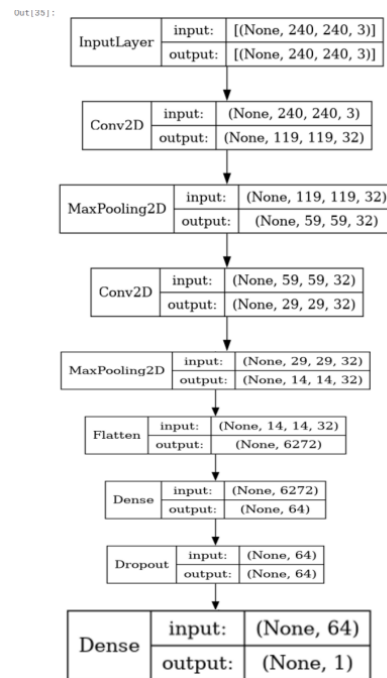
- model.summary(): Tóm tắt cấu trúc mô hình, Hiển thị thông tin chi tiết về mô hình đã xây dựng.

[(None, 240, 240, 3)]

- None đại diện cho batch size (batch: nhóm). Kích thước của mỗi batch có thể thay đổi tùy thuộc vào bộ nhớ của thiết bị và cách cài đặt của bạn. Vì vậy, kích thước batch đầu tiên (trục thứ nhất) thường được đặt là "None" để biểu thị rằng nó có thể thay đổi tùy ý.
- 240, 240: Kích thước 240x240 pixel
- 3: Mỗi pixel có 3 kênh màu (RGB: đỏ, xanh lá, xanh dương).

```
1 plot_model(model, show_shapes=True, show_layer_names=False)
```

- plot_model: Một hàm trong Keras giúp vẽ sơ đồ mô hình dưới dạng hình ảnh.
- model: Mô hình Keras muốn vẽ.
- show_shapes=True: Hiển thị kích thước đầu vào và đầu ra (shapes) của từng lớp.
- show_layer_names=False: Không hiển thị tên lớp trong sơ đồ. Nếu đặt là True, tên của từng lớp (ví dụ: conv2d, max_pooling2d) sẽ được hiển thị.



Hình 10: Kết quả trả về

Dense: Sau khi các lớp Conv2D và Pooling trích xuất đặc trưng từ ảnh, lớp Dense sẽ kết hợp các đặc trưng này lại để đưa ra các quyết định (như phân loại, dự đoán, v.v.).

```
1 visualekera.layered_view(model, legend=True)
```

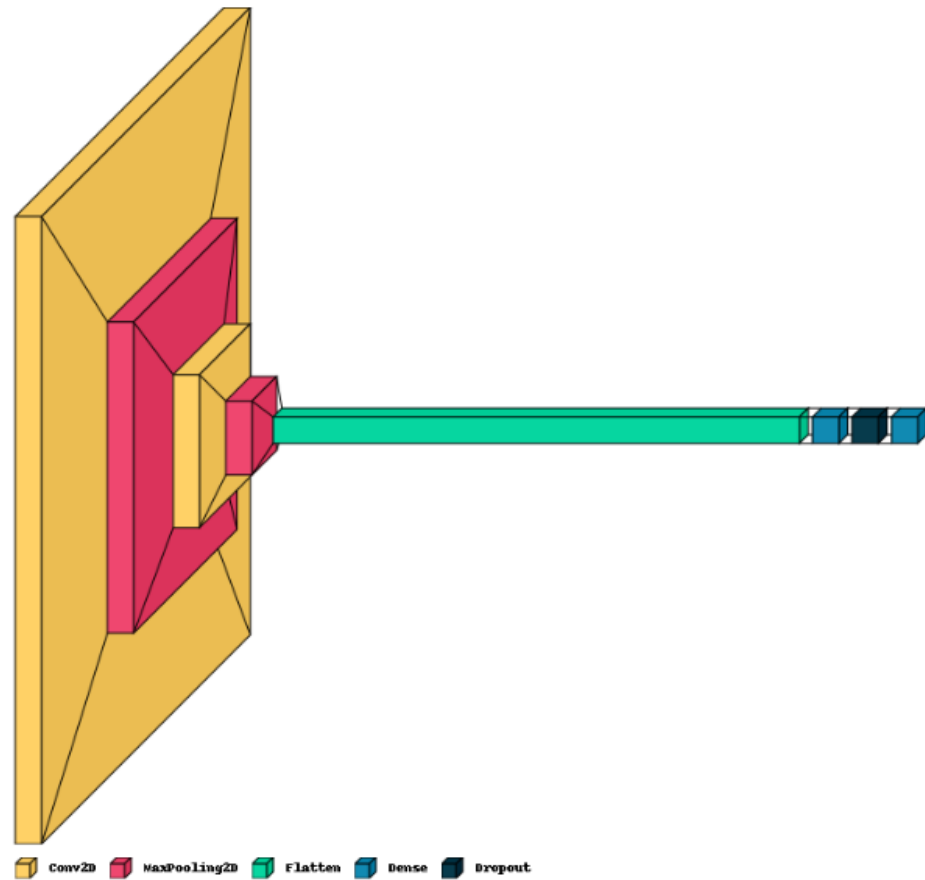
Tạo một sơ đồ trực quan thể hiện cấu trúc của mô hình.

-visualekera: thư viện Python hỗ trợ trực quan hóa mô hình Keras.

-layered_view: vẽ mô hình học sâu dưới dạng sơ đồ phân lớp, nơi mỗi lớp trong mô hình được biểu diễn như một hình khối. Các lớp này được kết nối với nhau để thể hiện cách chúng liên kết và truyền dữ liệu.

-legend=True: Tham số legend khi đặt là True sẽ hiển thị chú thích (legend) để giải thích các loại lớp trong mô hình (ví dụ: "Conv2D", "Dense", "MaxPooling2D", v.v.).

Out[36]:



Hình 11: Sơ đồ trực quan hóa mô hình CNN

```
1 class_weights = class_weight.compute_class_weight(class_weight = "
    balanced", classes= np.unique(y_train), y= y_train)
2 class_weights = dict(zip(np.unique(y_train), class_weights))
3 class_weights
```

- `class_weight.compute_class_weight`: Hàm được sử dụng để tính toán trọng số cho các lớp trong mô hình phân loại. Trọng số sẽ được tính sao cho lớp có ít mẫu sẽ có trọng số cao hơn và lớp có nhiều mẫu sẽ có trọng số thấp hơn, giúp mô hình không bị thiên lệch khi học.
- `class_weight = "balanced"`: Cài đặt này yêu cầu hàm tính toán trọng số theo phương pháp cân bằng. Điều này có nghĩa là các lớp với ít mẫu sẽ có trọng số cao hơn để bù đắp cho sự thiếu hụt trong quá trình huấn luyện.
- `classes= np.unique(y_train)`: Đây là danh sách các lớp (class labels) trong tập huấn luyện (`y_train`). `np.unique(y_train)` sẽ trả về tất cả các lớp duy nhất trong tập `y_train`.
- Hàm `compute_class_weight` sẽ tính toán trọng số cho từng lớp dựa trên số lượng mẫu của từng lớp trong `y_train`.
- `class_weights = dict(zip(np.unique(y_train), class_weights))`
+ `zip()`: kết hợp các nhãn lớp duy nhất và trọng số tương ứng thành các cặp.

+dict(): chuyển đổi các cặp này thành một từ điển, trong đó các nhãn lớp là khóa và các trọng số là giá trị.

```
{0: 1.0040160642570282, 1: 0.9960159362549801}
```

- class_weights: Sau khi chạy câu lệnh trên,
- class_weights sẽ là một từ điển chứa trọng số cho mỗi lớp trong tập huấn luyện.

```
1 history = model.fit(x_train,y_train,epochs = 100, class_weight=  
class_weights, validation_data=(x_test, y_test),verbose=1)
```

- epochs=100 chỉ định số vòng huấn luyện là 100. validation_data=(x_test, y_test) giúp theo dõi hiệu suất của mô hình trên dữ liệu kiểm tra sau mỗi epoch.
- verbose=1 cho phép hiển thị thông tin chi tiết về tiến trình huấn luyện.

4.5 Đánh giá mô hình CNN

```
1 plt.figure(figsize=(20,10))  
2 plt.subplot(1, 2, 1)  
3 plt.suptitle('Optimizer : Adam', fontsize=10)  
4 plt.ylabel('Loss', fontsize=16)  
5 plt.plot(history.history['loss'], label='Training Loss')  
6 plt.plot(history.history['val_loss'], label='Validation Loss')  
7 plt.legend(loc='upper right')  
8  
9 plt.subplot(1, 2, 2)  
10 plt.ylabel('Accuracy', fontsize=16)  
11 plt.plot(history.history['accuracy'], label='Training Accuracy')  
12 plt.plot(history.history['val_accuracy'], label='Validation Accuracy')  
13 plt.legend(loc='lower right')  
14 plt.show()
```

1. Tạo kích thước của hình ảnh

Hình ảnh có kích thước là 20x10 inch.

2. Vẽ biểu đồ đầu tiên (Loss):

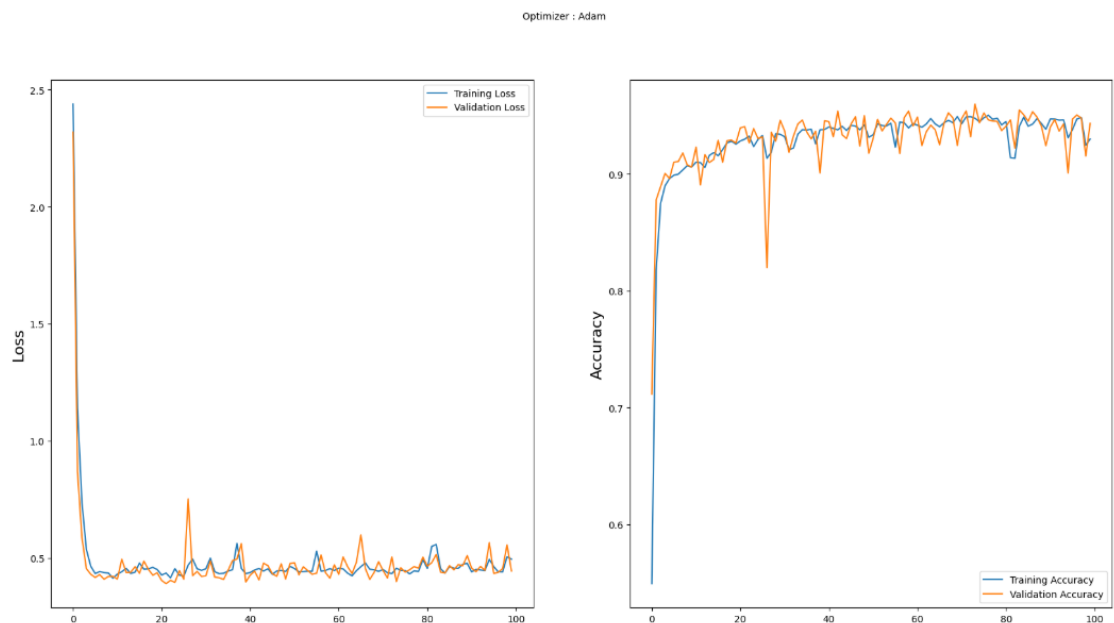
- plt.subplot(1, 2, 1): Tạo một khu vực vẽ đồ thị trong hình ảnh có 1 hàng và 2 cột (tức là 2 biểu đồ trong cùng một hàng). Biểu đồ đầu tiên sẽ ở vị trí (1, 2, 1).
- plt.suptitle('Optimizer : Adam', fontsize=10): Đặt tiêu đề chung cho toàn bộ hình ảnh, là "Optimizer: Adam".
- plt.ylabel('Loss', fontsize=16): Đặt nhãn cho trục y là "Loss".
- plt.plot(history.history['loss'], label='Training Loss'): Vẽ đồ thị Loss trong quá trình huấn luyện (training loss) từ history.history['loss'].
- plt.plot(history.history['val_loss'], label='Validation Loss'): Vẽ đồ thị Validation Loss từ history.history['val_loss'].
- plt.legend(loc='upper right'): Thêm legend (chú thích) ở góc trên bên phải để phân biệt giữa Training Loss và Validation Loss.

3. Vẽ biểu đồ thứ 2 (Accuracy)

- `plt.subplot(1, 2, 2)`: Tạo biểu đồ thứ hai trong khu vực vẽ 1 hàng và 2 cột.
- `plt.ylabel('Accuracy', fontsize=16)`: Đặt nhãn cho trục y là "Accuracy".
- `plt.plot(history.history['accuracy'], label='Training Accuracy')`: Vẽ đồ thị Accuracy trong quá trình huấn luyện từ `history.history['accuracy']`.
- `plt.plot(history.history['val_accuracy'], label='Validation Accuracy')`: Vẽ đồ thị **Validation Accuracy** từ `history.history['val_accuracy']`.
- `plt.legend(loc='lower right')`: Thêm legend (chú thích) ở góc dưới bên phải.

Các thông số trong `history.history`:

- `history.history['loss']`: Lưu trữ giá trị của loss trong mỗi epoch của quá trình huấn luyện.
- `history.history['val_loss']`: Lưu trữ giá trị của loss trong mỗi epoch của tập validation.
- `history.history['accuracy']`: Lưu trữ giá trị của accuracy trong mỗi epoch của quá trình huấn luyện.
- `history.history['val_accuracy']`: Lưu trữ giá trị của accuracy trong mỗi epoch của tập validation.



Đồ thị bên trái: Loss (Hàm mất mát)

- Training Loss (đường màu xanh) và Validation Loss (đường màu cam):
 - Ban đầu (epoch 0-20): Cả hai hàm mất mát giảm nhanh, cho thấy mô hình đang học tốt.

- Sau đó (epoch 20 trở đi):
 - + Cả hai giá trị dao động quanh một mức ổn định (gần 0.5). Điều này cho thấy mô hình đã hội tụ.
 - + Validation Loss dao động: Điều này có thể là do mô hình gặp dữ liệu đa dạng hơn ở tập validation hoặc các yếu tố ngẫu nhiên trong huấn luyện.

Kết luận từ Loss:

- Sự gần gũi giữa Training Loss và Validation Loss cho thấy mô hình không bị overfitting.
- Validation Loss dao động nhưng không tăng lên nhiều, cho thấy mô hình duy trì được khả năng tổng quát hóa.

Đồ thị bên phải: Accuracy (Độ chính xác)

- Training Accuracy (đường màu xanh) và Validation Accuracy (đường màu cam):
 - Ban đầu (epoch 0-20): Cả hai giá trị tăng nhanh, điều này phù hợp với sự giảm của Loss.
 - Sau đó (epoch 20 trở đi):
 - + Accuracy đạt trên 90%: Đây là một hiệu suất tốt cho bài toán.
 - + Validation Accuracy dao động nhẹ: Điều này là bình thường và có thể liên quan đến sự phức tạp của dữ liệu.

Kết luận từ Accuracy:

- Mô hình học tốt cả trên tập huấn luyện và kiểm tra, vì độ chính xác của cả hai tập rất gần nhau.
- Mô hình có khả năng tổng quát hóa tốt vì không có sự chênh lệch lớn giữa Training Accuracy và Validation Accuracy.

Note:

Validation (xác thực) trong học máy (Machine Learning) là quá trình đánh giá hiệu suất của một mô hình trên một tập dữ liệu không được sử dụng trong quá trình huấn luyện, nhưng cũng không phải tập kiểm tra (test set). Mục đích chính là đo lường mức độ tổng quát hóa (generalization) của mô hình đối với dữ liệu mà nó chưa từng thấy.

Overfitting (quá khớp) là một vấn đề phổ biến trong học máy (machine learning), xảy ra khi mô hình học quá kỹ các chi tiết hoặc nhiễu (noise) trong dữ liệu huấn luyện đến mức nó không thể tổng quát hóa tốt cho dữ liệu mới. Nói cách khác, mô hình hoạt động rất tốt trên dữ liệu huấn luyện nhưng lại có hiệu suất kém trên dữ liệu kiểm tra hoặc dữ liệu chưa từng thấy.

4. Đánh giá mô hình

Đoạn mã này sẽ tính toán độ chính xác của mô hình trên tập kiểm tra (x_{test} và y_{test}), rồi in ra kết quả dưới dạng phần trăm.

```
1 results = model.evaluate(x_test, y_test)
2 print('The current model achieved an accuracy of {}%'.format(round(results
    [1]*100,2)))
```

```
188/188 [=====] - 1s 6ms/step - loss: 0.4449 - accuracy: 0.9433  
The current model achieved an accuracy of 94.33%!
```

`results = model.evaluate(x_test, y_test):`

- Hàm `evaluate()` dùng để đánh giá mô hình trên tập kiểm tra.
- `x_test`: Dữ liệu đầu vào của tập kiểm tra.
- `y_test`: Nhãn đúng của dữ liệu đầu vào trong tập kiểm tra.
- Hàm này sẽ trả về một danh sách, trong đó phần tử đầu tiên là `loss` (mức độ sai số của mô hình) và phần tử thứ hai là `accuracy` (độ chính xác của mô hình).

```
1 # compute predictions  
2 predictions = model.predict(x_test)  
3 y_pred = []  
4 for i in predictions:  
5     if i >= 0.5:  
6         y_pred.append(1)  
7     else:  
8         y_pred.append(0)
```

Đoạn mã ở trên thực hiện việc **dự đoán** đầu ra của mô hình cho tập kiểm tra `x_test` và sau đó **chuyển đổi** các giá trị dự đoán thành nhãn 0 hoặc 1 (chuyển thành dạng phân loại nhị phân) dựa trên ngưỡng 0.5.

`model.predict(x_test)`: Dự đoán xác suất cho mỗi mẫu trong tập kiểm tra, Hàm `predict()` được sử dụng để dự đoán nhãn cho tập kiểm tra `x_test`. Kết quả trả về sẽ là một mảng (array) có cùng số lượng phần tử với `x_test`. Mỗi phần tử trong `predictions` là một giá trị xác suất (từ 0 đến 1) cho mỗi mẫu trong tập kiểm tra, do mô hình trả về khi sử dụng hàm kích hoạt **sigmoid** trong lớp đầu ra. Mỗi giá trị này đại diện cho xác suất thuộc lớp 1.

Mảng `predictions` chứa các giá trị xác suất từ 0 đến 1, và bạn muốn chuyển đổi chúng thành nhãn 0 hoặc 1.

Ngưỡng 0.5: Đoạn mã này so sánh mỗi giá trị trong `predictions` với ngưỡng 0.5.

- Nếu giá trị xác suất lớn hơn hoặc bằng 0.5, mẫu đó sẽ được gán nhãn 1 (dự đoán là lớp 1).
- Nếu giá trị xác suất nhỏ hơn 0.5, mẫu đó sẽ được gán nhãn 0 (dự đoán là lớp 0).

`y_pred.append(1)` và `y_pred.append(0)`: Sau khi so sánh, nhãn 1 hoặc 0 sẽ được thêm vào danh sách `y_pred`, tạo thành một danh sách chứa các nhãn phân loại của các mẫu trong tập kiểm tra.

Đoạn mã trên có thể được thay thế hoặc rút gọn với `numpy` để làm nhanh hơn:

```
1 predictions = model.predict(x_test)
2 y_pred = (predictions >= 0.5).astype(int)
```

Đây cách tối ưu để chuyển đổi các giá trị xác suất trong *predictions* thành nhãn 0 hoặc 1 mà không cần phải sử dụng vòng lặp for. Đây là cách sử dụng Numpy để xử lý một cách hiệu quả.

- **predictions >= 0.5**: Đây là một phép so sánh vector hóa. Numpy sẽ so sánh tất cả các giá trị trong mảng predictions với 0.5 và tạo ra một mảng Boolean (True nếu giá trị >= 0.5, False nếu giá trị < 0.5).
 - Ví dụ: nếu predictions = [0.8, 0.2, 0.9, 0.4], thì predictions >= 0.5 sẽ trả về [True, False, True, False].
- **.astype(int)**: Phương thức này chuyển đổi các giá trị Boolean True/False thành các số nguyên 1/0.
 - True sẽ trở thành 1 và False sẽ trở thành 0.
 - Ví dụ: [True, False, True, False] sẽ được chuyển thành [1, 0, 1, 0].

Kết quả:

- y_pred sẽ là mảng chứa nhãn phân loại với giá trị 0 hoặc 1, tùy thuộc vào giá trị xác suất của mỗi mẫu.

5. Vẽ ma trận nhầm lẫn (Confusion Matrix)

```
1 import matplotlib.pyplot as plt
2 import seaborn as sns
3
4 # Compute confusion matrix
5 cnf_matrix = confusion_matrix(y_test, y_pred)
6
7 # Normalize for better visualization
8 cnf_matrix_norm = cnf_matrix.astype('float') / cnf_matrix.sum(axis=1)[:, np.
    newaxis]
9
10 # Employ a triangular structure for visual clarity
11 mask = np.triu(np.ones_like(cnf_matrix_norm, dtype=bool))
12
13 # Create a visually distinct heatmap with a different colormap
14 sns.heatmap(cnf_matrix_norm, mask=mask, annot=True, fmt='.2f', cmap='coolwarm',
    vmin=0, vmax=1,
    linewidths=0.5, cbar_kws={'label': 'Normalized Values'})
15
16
17 # Customize labels and title
18 plt.xlabel('Predicted Label')
19 plt.ylabel('True Label')
20 plt.title('Confusion Matrix (Triangular Structure)')
21
22 plt.show()
```

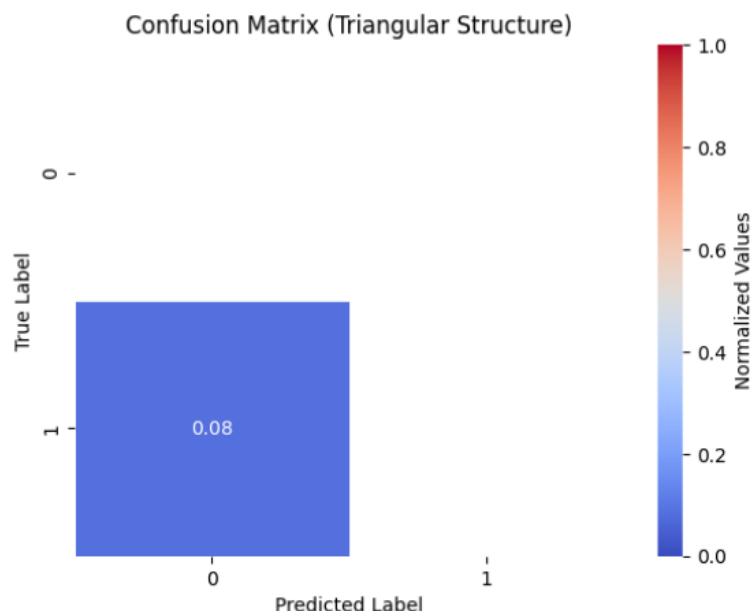
Đoạn mã sẽ vẽ ma trận nhầm lẫn (confusion matrix) với các tùy chọn định dạng đẹp mắt và dễ hiểu. Cụ thể, nó sẽ vẽ một heatmap của ma trận nhầm lẫn đã được chuẩn hóa (normalized) để dễ dàng so sánh giữa các lớp dự đoán và lớp thực tế.

- Tính toán ma trận nhầm lẫn:
Hàm confusion_matrix(y_test, y_pred) từ thư viện sklearn.metrics tính toán ma trận nhầm lẫn giữa nhãn thực tế (y_test) và nhãn dự đoán (y_pred). Ma trận nhầm lẫn cho biết số lượng dự đoán đúng và sai cho từng lớp trong mô hình phân loại.

- Chuẩn hóa ma trận nhầm lẫn:
Phép chuẩn hóa này chia từng phần tử của ma trận nhầm lẫn cho tổng các giá trị trong từng hàng (tức là tổng số mẫu thực tế của từng lớp). Kết quả là ma trận xác suất, giúp dễ dàng so sánh hiệu suất mô hình cho từng lớp.
- Tạo mặt nạ tam giác trên heatmap:
np.triu() tạo ra một mặt nạ tam giác (triangle mask), giữ lại phần trên của ma trận, bỏ qua phần dưới. Điều này giúp loại bỏ các giá trị không cần thiết trong heatmap để tăng tính dễ đọc.
- Vẽ heatmap với seaborn(*Seaborn* là một thư viện Python được xây dựng dựa trên Matplotlib, chuyên dùng để trực quan hóa dữ liệu. Nó giúp tạo ra các biểu đồ đẹp, dễ tùy chỉnh, và tích hợp tốt với *andas DataFrame*, giúp bạn làm việc với dữ liệu tabular dễ dàng hơn.)

sns.heatmap(): Dùng để vẽ heatmap của ma trận nhầm lẫn.

- **mask=mask**: Áp dụng mặt nạ tam giác để làm cho heatmap chỉ hiển thị phần trên của ma trận.
- **annot=True**: Hiển thị giá trị của từng phần tử trong ma trận.
- **fmt='.2f'**: Định dạng các giá trị hiển thị với 2 chữ số thập phân.
- **cmap='coolwarm'**: Chọn màu sắc cho heatmap, với màu lạnh cho các giá trị thấp và màu ấm cho các giá trị cao.
- **vmin=0, vmax=1**: Đặt giá trị cực tiểu và cực đại của color scale từ 0 đến 1, vì ma trận đã được chuẩn hóa.
- **linewidths=0.5**: Đặt độ dày của đường viền giữa các ô trong heatmap.
- **cbar_kws='label': 'Normalized Values'**: Gán nhãn cho thanh màu bên cạnh.



Confusion Matrix là một công cụ được sử dụng để đánh giá hiệu suất của mô hình phân loại. Ma trận này so sánh giá trị thực (**True Labels**) với giá trị dự đoán (**Predicted Labels**).

- **Trục X (Predicted Label):** Là các nhãn được mô hình dự đoán.
- **Trục Y (True Label):** Là nhãn thực tế (*ground truth*).

Trong hình:

- **Label 0:** Đại diện cho lớp “Negative”.
- **Label 1:** Đại diện cho lớp “Positive”.

***Dữ liệu trong ma trận (giá trị 0.08):**

Giá trị này cho biết phần trăm mẫu dữ liệu thuộc một ô cụ thể. Tuy nhiên:

- Do đoạn mã có sử dụng `mask tam giác (np.triu)`, chỉ hiển thị các giá trị nằm trong phần tam giác trên của ma trận.
- Các ô không nằm trong tam giác trên (bao gồm cả trục chính) đã bị ẩn đi.

Ví dụ:

Giá trị 0.08 ở ô (1, 0) đại diện cho 8% dữ liệu thật sự thuộc lớp **1 (Positive)** nhưng được dự đoán nhầm thành lớp **0 (Negative)**.

***Màu sắc:**

- Bảng màu (`cmap='coolwarm'`) chuyển từ xanh lam (giá trị thấp) đến đỏ (giá trị cao).
- Giá trị màu sắc biểu diễn mức độ lớn nhỏ của giá trị trong ma trận.
 - **Màu xanh:** Giá trị thấp.
 - **Màu đỏ:** Giá trị cao.

***Thanh chuẩn hóa (Colorbar):**

Thanh bên phải hiển thị giá trị màu tương ứng (Normalized Values từ 0 đến 1). Các giá trị trong ma trận đã được chuẩn hóa bằng cách chia cho tổng số mẫu trên mỗi hàng (`cnf_matrix.sum(axis=1)`).

***Tiêu đề và trục:**

- **Tiêu đề (“Confusion Matrix - Triangular Structure”):** Cho biết đây là biểu diễn của ma trận nhầm lẫn với cấu trúc tam giác.
- **Labels trên trục X và Y:** Hiển thị “Predicted Label” và “True Label” để rõ ràng mục đích của từng trục.

Một ma trận nhầm lẫn dạng khác là: Ma trận nhầm lẫn - Nhấn mạnh hình lục giác (Confusion Matrix - Hexagonal Emphasis)

```
1 import matplotlib.pyplot as plt
2 import seaborn as sns
3 import numpy as np
4
5 # Compute confusion matrix
6 cnf_matrix = confusion_matrix(y_test, y_pred)
7
8 # Create a hexagonal mask for a visually striking shape
9 mask = np.triu(np.ones_like(cnf_matrix, dtype=bool)) & np.tril(np.ones_like(
    cnf_matrix, dtype=bool), k=-1)
10
11 # Plot using Seaborn with distinct customizations
12 sns.heatmap(
13     cnf_matrix,
14     annot=True,
15     fmt='.2f',
16     cmap='cubehelix', # Employ a vibrant colormap
17     mask=mask,
18     linewidths=0.5,
19     cbar=False,
20     xticklabels=["Negative", "Positive"],
21     yticklabels=["Negative", "Positive"]
22 )
23 plt.title('Confusion Matrix - Hexagonal Emphasis')
24 plt.xlabel('Predicted Label')
25 plt.ylabel('True Label')
26 plt.show()
```

1. Tính toán ma trận nhầm lẫn:

- `confusion_matrix(y_test, y_pred)`: Tính toán ma trận nhầm lẫn giữa nhãn thực tế (`y_test`) và nhãn dự đoán (`y_pred`).

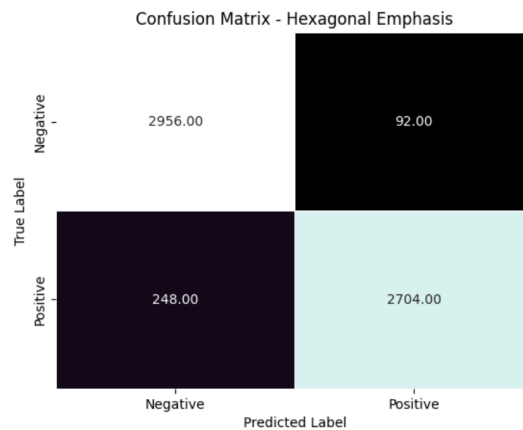
Tạo mặt nạ (mask) hình lục giác:

- `np.triu()`: Tạo mặt nạ tam giác trên của ma trận (giữ lại phần trên, bỏ qua phần dưới).
- `np.tril(k=-1)`: Tạo mặt nạ tam giác dưới của ma trận, nhưng chỉ giữ lại phần phía dưới đường chéo chính (không bao gồm các phần tử trên đường chéo chính).
- `&`: Kết hợp hai mặt nạ bằng phép toán logic AND, tạo ra một mặt nạ lục giác nơi chỉ giữ lại các phần tử nằm trên hoặc dưới đường chéo chính.

Vẽ heatmap của ma trận nhầm lẫn:

- `sns.heatmap()`: Dùng để vẽ heatmap của ma trận nhầm lẫn.
- `annot=True`: Hiển thị giá trị trong các ô của ma trận.
- `fmt='.2f'`: Định dạng các giá trị với 2 chữ số thập phân.
- `cmap='cubehelix'`: Sử dụng bảng màu `cubehelix` với các sắc độ mạnh, tạo sự nổi bật cho các giá trị.

- `mask=mask`: Áp dụng mặt nạ đã tạo ra trước đó (giúp tạo hình lục giác).
- `linewidths=0.5`: Độ dày của đường viền giữa các ô trong ma trận.
- `cbar=False`: Tắt thanh màu (colorbar) bên cạnh.
- `xticklabels` và `yticklabels`: Gán nhãn cho các trục x và y (là **Negative** và **Positive**).



***Cấu trúc của ma trận nhầm lẫn:**

- **Trục hoành (Predicted Label)**: Dự đoán của mô hình.
 - **Negative**: Nhãn dự đoán là 0.
 - **Positive**: Nhãn dự đoán là 1.
- **Trục tung (True Label)**: Nhãn thực tế của dữ liệu.
 - **Negative**: Nhãn thực tế là 0.
 - **Positive**: Nhãn thực tế là 1.

***Các giá trị trong ma trận:**

Ma trận bao gồm 4 ô, mỗi ô tương ứng với một loại kết quả:

- **[2956] True Negative (TN)**: Số lượng mẫu thực tế là **Negative** (0) và được dự đoán chính xác là **Negative** (0).
- **[92] False Positive (FP)**: Số lượng mẫu thực tế là **Negative** (0) nhưng bị mô hình dự đoán sai thành **Positive** (1).
- **[248] False Negative (FN)**: Số lượng mẫu thực tế là **Positive** (1) nhưng bị mô hình dự đoán sai thành **Negative** (0).
- **[2704] True Positive (TP)**: Số lượng mẫu thực tế là **Positive** (1) và được dự đoán chính xác là **Positive** (1).

Ý nghĩa trực quan từ hình ảnh:

- **Màu sắc**:
 - **Màu xanh nhạt (e.g., 2704)**: Giá trị cao hơn so với các ô khác, thể hiện số lượng dự đoán đúng cho lớp **Positive**.

- **Màu tối (e.g., 248):** Giá trị thấp hơn, thể hiện số lỗi **False Negative**.
- **Mask (hình tam giác):**
 - Đoạn mã đã áp dụng một mặt nạ để hiển thị dữ liệu trong cấu trúc tam giác, nhằm làm nổi bật một phần ma trận.

Đánh giá mô hình qua từng chỉ số

(a) Accuracy (Độ chính xác)

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Với các giá trị trong bài:

$$\text{Accuracy} = \frac{2956 + 2704}{2956 + 92 + 248 + 2704} = \frac{5660}{6000} = 0.9433 (94.33\%)$$

- Ý nghĩa: 94.33% dự đoán của mô hình là chính xác, tức là mô hình hoạt động tốt trên toàn bộ tập dữ liệu (bao gồm cả lớp "Negative" và "Positive").
- Hạn chế: Accuracy có thể không phản ánh đúng hiệu quả nếu tập dữ liệu mất cân bằng (khi một lớp xuất hiện nhiều hơn lớp còn lại). Vì vậy, cần kết hợp các chỉ số khác để đánh giá mô hình chính xác hơn.

(b) Precision (Độ chính xác cho lớp Positive)

$$\text{Precision} = \frac{TP}{TP + FP}$$

Với các giá trị trong bài:

$$\text{Precision} = \frac{2704}{2704 + 92} = \frac{2704}{2796} = 0.967 (96.7\%)$$

- Ý nghĩa: Khi mô hình dự đoán một mẫu thuộc lớp Positive, có 96.7% khả năng dự đoán đó là đúng. Điều này cho thấy mô hình rất ít mắc lỗi False Positive (dự đoán nhầm mẫu Negative thành Positive).
- Thích hợp khi nào?
Precision đặc biệt quan trọng khi chi phí của False Positive cao. Ví dụ:
 - + Trong chẩn đoán bệnh hiểm, không nên dự đoán nhầm người không mắc bệnh là có bệnh (False Positive) vì điều này gây lo lắng và tốn kém.

(c) Recall (Sensitivity - Độ nhạy)

$$\text{Recall} = \frac{TP}{TP + FN}$$

Với các giá trị trong bài:

$$\text{Recall} = \frac{2704}{2704 + 248} = \frac{2704}{2952} = 0.916 (91.6\%)$$

- Ý nghĩa: Trong tất cả các mẫu thực sự thuộc lớp Positive, mô hình nhận diện được 91.6%.
Điều này thể hiện mô hình có khả năng phát hiện mẫu Positive tốt, nhưng vẫn còn sót một số lượng nhỏ False Negative (dự đoán nhầm Positive thành Negative).
- Thích hợp khi nào? Recall quan trọng khi cần giảm thiểu lỗi False Negative, ví dụ: oTrong y tế, bỏ sót người mắc bệnh (False Negative) có thể gây hậu quả nghiêm trọng.

(d) F1-Score

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

- Ý nghĩa:
F1-Score là trung bình điều hòa của Precision và Recall, cân bằng giữa việc dự đoán đúng và khả năng phát hiện mẫu Positive.

Với giá trị cao (94.1%), mô hình có sự cân đối tốt giữa Precision và Recall.

Với các giá trị tính từ trên:

$$F1 = 2 \times \frac{0.967 \times 0.916}{0.967 + 0.916} = 0.941 \text{ (94.1\%)}$$

Nhận xét về mô hình

- (a) **Hiệu quả chung:** Mô hình hoạt động tốt với độ chính xác tổng thể (Accuracy) cao (94.33%).
- (b) **Điểm mạnh:**
 - Precision cao (96.7%): Rất ít lỗi False Positive. Mô hình dự đoán lớp Positive một cách đáng tin cậy.
 - Recall tương đối cao (91.6%): Phát hiện được hầu hết các mẫu thực sự thuộc lớp Positive.
- (c) **Hạn chế:** Mô hình vẫn bỏ sót một số lượng nhỏ mẫu Positive (248 False Negative). Nếu trong bài toán thực tế, lỗi bỏ sót là quan trọng, cần cải thiện Recall.

Kết luận về mô hình

Dựa trên các chỉ số này, ta có thể kết luận:

- Mô hình phù hợp với bài toán hiện tại, đặc biệt khi bạn cần dự đoán chính xác các mẫu Positive (Precision cao).
- Tuy nhiên, nếu bài toán yêu cầu phát hiện toàn bộ mẫu Positive (ưu tiên Recall), thì mô hình vẫn có thể cải thiện để giảm thiểu lỗi False Negative.

5 LỜI KẾT

Trong báo cáo này, chúng em đã nghiên cứu và triển khai một phương pháp sử dụng mạng nơ-ron tích chập (CNN) nhằm phát hiện vết nứt trên bề mặt xi măng thông qua hình ảnh. Với sự hỗ trợ của công nghệ học sâu và khả năng xử lý dữ liệu hình ảnh hiệu quả của mạng CNN, hệ thống đã đạt được kết quả khả quan trong việc phân loại và phát hiện vết nứt, từ đó cung cấp một giải pháp tự động, chính xác cho việc kiểm tra chất lượng bề mặt xi măng.

Kết quả thực nghiệm đã cho thấy mô hình có khả năng nhận diện vết nứt với độ chính xác cao, nhờ vào việc tối ưu hóa các tham số mô hình và sử dụng kỹ thuật xử lý dữ liệu thích hợp. Tuy nhiên, hệ thống vẫn tồn tại một số hạn chế như khả năng phát hiện trên các hình ảnh có điều kiện ánh sáng phức tạp hoặc chứa nhiễu lớn.

Trong tương lai, để cải thiện hệ thống, các nghiên cứu tiếp theo có thể tập trung vào:

- Sử dụng các kiến trúc CNN tiên tiến hơn, chẳng hạn như ResNet, EfficientNet hoặc các mạng tích hợp kỹ thuật Attention.
- Mở rộng và đa dạng hóa tập dữ liệu để tăng khả năng tổng quát hóa của mô hình.
- Áp dụng các kỹ thuật tăng cường dữ liệu (data augmentation) và chuyển học (transfer learning) để cải thiện hiệu suất trên các tập dữ liệu nhỏ hoặc không đồng nhất.

Phát hiện vết nứt trên bề mặt xi măng là một bước quan trọng trong việc đảm bảo chất lượng công trình và giảm thiểu chi phí bảo trì. Chúng em hy vọng rằng phương pháp được trình bày trong nghiên cứu này sẽ đóng góp vào sự phát triển của các giải pháp kiểm tra tự động, góp phần nâng cao hiệu quả và độ tin cậy trong lĩnh vực xây dựng.

Tài liệu

- [1] Code from Kaggle <https://www.kaggle.com/code/tahmidmir/pavement-crack-inspection-cnn-vit>
- [2] Kuo, C. (2016). Understanding convolutional neural networks with a mathematical model. ArXiv, abs/1609.04112. <https://doi.org/10.1016/j.jvcir.2016.11.003>.
- [3] Mahmoudi, A., Chetouani, A., Boufera, F., & Tabia, H. (2020). Learnable pooling weights for facial expression recognition. Pattern Recognit. Lett., 138, 644-650. <https://doi.org/10.1016/j.patrec.2020.09.001>.
- [4] Sun, M., Song, Z., Jiang, X., Pan, J., Pang, Y. (2017). Learning Pooling for Convolutional Neural Network. Neurocomputing, 224, 96-104. <https://doi.org/10.1016/j.neucom.2016.10.049>.
- [5] Nguyen, T. T. (2020). Deep Learning Cơ Bản (2nd ed.).
- [6] Yamashita, R., Nishio, M., , R., Togashi, K. (2018). Convolutional neural networks: an overview and application in radiology. Insights into Imaging, 9, 611 - 629. <https://doi.org/10.1007/s13244-018-0639-9>.
- [7] Sarigul, M., Özyildirim, B., Avci, M. (2019). Differential convolutional neural network. Neural networks : the official journal of the International Neural Network Society, 116, 279-287 . <https://doi.org/10.1016/j.neunet.2019.04.025>.
- [8] Li, Z., Liu, F., Yang, W., Peng, S., Zhou, J. (2020). A Survey of Convolutional Neural Networks: Analysis, Applications, and Prospects. IEEE Transactions on Neural Networks and Learning Systems, 33, 6999-7019. <https://doi.org/10.1109/TNNLS.2021.3084827>.
- [9] Bernal, J., Kushibar, K., Asfaw, D., Valverde, S., Oliver, A., Martí, R., Lladó, X. (2017). Deep convolutional neural networks for brain image analysis on magnetic resonance imaging: a review. Artificial intelligence in medicine, 95, 64-81 . <https://doi.org/10.1016/j.artmed.2018.08.008>.
- [10] Gu, J., Wang, Z., Kuen, J., , L., Shahroudy, A., Shuai, B., Liu, T., Wang, X., Wang, G., Cai, J., Chen, T. (2015). Recent advances in convolutional neural networks. ArXiv, abs/1512.07108. <https://doi.org/10.1016/J.PATCOG.2017.10.013>.
- [11] Chen, L., Li, S., Bai, Q., Yang, J., Jiang, S., Miao, Y. (2021). Review of Image Classification Algorithms Based on Convolutional Neural Networks. Remote. Sens., 13, 4712. <https://doi.org/10.3390/rs13224712>.
- [12] Yu, S., Jia, S., Xu, C. (2017). Convolutional neural networks for hyperspectral image classification. Neurocomputing, 219, 88-98. <https://doi.org/10.1016/j.neucom.2016.09.010>.
- [13] Pelt, D., Sethian, J. (2017). A mixed-scale dense convolutional neural network for image analysis. Proceedings of the National Academy of Sciences, 115, 254 - 259. <https://doi.org/10.1073/pnas.1715832114>.
- [14] Cheng, P., Malhi, H. (2017). Transfer Learning with Convolutional Neural Networks for Classification of Abdominal Ultrasound Images. Journal of Digital Imaging, 30, 234-243. <https://doi.org/10.1007/s10278-016-9929-2>.



- [15] Niu, X., Suen, C. (2012). A novel hybrid CNN-SVM classifier for recognizing handwritten digits. Pattern Recognit., 45, 1318-1325. <https://doi.org/10.1016/j.patcog.2011.09.021>.
- [16] Data Wow' Blog in Thailand Interns Explain CNN - Data Wow blog – Data Science Consultant Thailand | Data Wow in Bangkok
- [17] Deep Learning course at Stanford University CS 230 - Mạng nơ ron tích chập cheatsheet CS231n Convolutional Neural Networks for Visual Recognition
- [18] Wikipedia article in Kernel (image proccesing) [https://en.wikipedia.org/wiki/Kernel_\(image_processing\)](https://en.wikipedia.org/wiki/Kernel_(image_processing))
- [19] Deep Learning Ian Goodfellow, Yoshua Bengio và Aaron Courville
- [20] Tham khảo ChatGPT với một số đoạn code